

Curso Profesional de Entity Framework Core 8

Módulo 5 - Persistencia empresarial

PRÁCTICAS

AUTOR: JAIME GALLO

.NET 8 · Entity Framework Core 8 · Visual Studio Community · SQL Server LocalDB · Proyecto AceriaData

Índice

Punto 5.1 — Concurrencia optimista: concepto y necesidad	3
Punto 5.2 — Configuración de tokens de concurrencia	8
Punto 5.3 — Resolución de conflictos de concurrencia	14
Punto 5.4 — Transacciones: SaveChanges y transacciones explícitas	21
Punto 5.5 — Transacciones ambientales y buenas prácticas	26
Punto 5.6 — Migraciones en entornos de producción: estrategias y despliegue	32
Punto 5.7 — Migraciones idempotentes y scripts SQL	37
Punto 5.8 — Migraciones en equipos: conflictos y buenas prácticas	40
Punto 5.9 — Patrón Repositorio y Unidad de Trabajo en aplicaciones empresariales	44
Punto 5.10 — Logging y diagnóstico en Entity Framework Core	49
Punto 5.11 — Testing con EF Core	55
Punto 5.12 — Buenas prácticas y anti-patronos en persistencia empresarial	63

Curso Profesional de Entity Framework Core 8

Módulo 5 — Prácticas: Persistencia empresarial, concurrencia, transacciones, despliegue, testing y buenas prácticas

Autor: JAIME GALLO

Cada práctica trabaja sobre un estado completo y ejecutable de AceriaData. La secuencia es acumulativa desde M04/PROYECTO/4.12: cada punto conserva todo lo incorporado anteriormente y añade únicamente lo necesario para el nuevo contenido.

Entorno de trabajo: .NET 8 · C# 12 · Entity Framework Core 8 · SQL Server Express LocalDB · Visual Studio Community · PowerShell para los scripts de despliegue.

Punto 5.1 — Concurrencia optimista: concepto y necesidad

Contexto del proyecto

Este punto continúa directamente M04/PROYECTO/4.12. La carpeta M05/PROYECTO/5.1 contiene la solución completa AceriaData.sln y conserva las capas y migraciones existentes. El modelo no cambia todavía; se conserva la historia heredada del Módulo 4.

Objetivo práctico

Reproducir una actualización perdida real sobre la misma propiedad y contrastarla con cambios concurrentes sobre propiedades distintas.

Paso 1: Abrir el estado completo del punto

```
cd M05/PROYECTO/5.1
dotnet restore AceriaData.sln
```

POWERSHELL

La restauración se ejecuta sobre la solución local del punto. No es necesario copiar archivos desde otros puntos: la carpeta ya contiene el estado acumulativo completo.

Paso 2: Identificar los archivos principales

En este punto se trabajan principalmente estos archivos:

- src/AceriaData.Infrastructure/Repositories/ConcurrenciaOptimistaM5Repositorio.cs
- src/AceriaData.Application/ConcurrenciaOptimistaM5UseCase.cs

Los archivos se estudian dentro de su capa real. Application conserva sus abstracciones y casos de uso; Infrastructure contiene EF Core y acceso a proveedor; Console compone y ejecuta; los tests se ubican fuera de las capas de producción cuando aparecen en 5.9 y 5.11.

Paso 3: Implementar y estudiar ConcurrenciaOptimistaM5Repositorio.cs

```
public ConcurrenciaMismaPropiedadDto DemostrarActualizacionPerdidaMismaPropiedad()
{
    RestaurarEstadoInicial();
}
```

CSHARP

```

using var scopeA = _scopeFactory.CreateScope();
using var scopeB = _scopeFactory.CreateScope();

var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();
var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();

SqlCommandCounterInterceptor.Instance.Reset();

var ordenA = contextA.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);
var ordenB = contextB.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);

var clienteInicial = ordenA.Cliente;

ordenA.Cliente = "Cliente actualizado por A";
contextA.SaveChanges();

ordenB.Cliente = "Cliente actualizado por B";
contextB.SaveChanges();

using var scopeVerificacion = _scopeFactory.CreateScope();
var contextVerificacion = scopeVerificacion.ServiceProvider.GetRequiredService<AceriaDbContext>();
var clienteFinal = contextVerificacion.OrdenesFabricacion
    .AsNoTracking()
    .Where(o => o.NumeroOrden == NumeroOrden)
    .Select(o => o.Cliente)
    .Single();

return new ConcurrencyMismaPropiedadDto(
    clienteInicial,
    ordenA.Cliente,
    ordenB.Cliente,
    clienteFinal,
    clienteFinal == ordenB.Cliente && clienteFinal != ordenA.Cliente,
    SqlCommandCounterInterceptor.Instance.SnapshotCommands());
}

```

Explicación línea a línea — ConcurrencyOptimistaM5Repositorio.cs

Línea 1	<code>public ConcurrencyMismaPropiedadDto DemostrarActualizacionPerdidaMismaPropiedad()</code> → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 2	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 3	<code>RestaurarEstadoInicial();</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 5	<code>using var scopeA = _scopeFactory.CreateScope();</code> → Crea un ámbito de DI independiente; los servicios Scoped, incluido <code>AceriaDbContext</code> , no se comparten con otros actores.
Línea 6	<code>using var scopeB = _scopeFactory.CreateScope();</code> → Crea un ámbito de DI independiente; los servicios Scoped, incluido <code>AceriaDbContext</code> , no se comparten con otros actores.
Línea 8	<code>var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve el <code>DbContext</code> del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.
Línea 9	<code>var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve el <code>DbContext</code> del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.
Línea 11	<code>SqlCommandCounterInterceptor.Instance.Reset();</code> → Reinicia el contador para que solo se midan los comandos de este escenario.
Línea 13	<code>var ordenA = contextA.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 14	<code>var ordenB = contextB.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 16	<code>var clienteInicial = ordenA.Cliente;</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 18	<code>ordenA.Cliente = "Cliente actualizado por A";</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 19	<code>contextA.SaveChanges();</code> → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.
Línea 21	<code>ordenB.Cliente = "Cliente actualizado por B";</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 22	<code>contextB.SaveChanges();</code> → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.
Línea 24	<code>using var scopeVerificacion = _scopeFactory.CreateScope();</code> → Crea un ámbito de DI independiente; los servicios Scoped, incluido <code>AceriaDbContext</code> , no se comparten con otros actores.
Línea 25	<code>var contextVerificacion = scopeVerificacion.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve el <code>DbContext</code> del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.
Línea 26	<code>var clienteFinal = contextVerificacion.OrdenesFabricacion</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 27	<code>.AsNoTracking()</code> → Ejecuta una lectura sin dejar entidades en el <code>ChangeTracker</code> .
Línea 28	<code>.Where(o => o.NumeroOrden == NumeroOrden)</code> → Añade el predicado a la consulta; EF Core intentará traducirlo al proveedor antes de materializar.
Línea 29	<code>.Select(o => o.Cliente)</code> → Proyecta únicamente la forma de datos necesaria para el resultado.
Línea 30	<code>.Single();</code> → Materializa o agrega la consulta en este punto, provocando la ejecución contra el proveedor cuando procede.
Línea 32	<code>return new ConcurrenciaMismaPropiedadDto(</code> → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.
Línea 33	<code>clienteInicial,</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 34	<code>ordenA.Cliente,</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 35	<code>ordenB.Cliente,</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 36	<code>clienteFinal,</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 37	<code>clienteFinal == ordenB.Cliente && clienteFinal != ordenA.Cliente,</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 38	<code>SqlCommandCounterInterceptor.Instance.SnapshotCommands();</code> → Recupera los comandos SQL realmente interceptados durante la operación.
Línea 39	<code>}</code> → Delimita el bloque sintáctico correspondiente.

Paso 4: Implementar y estudiar ConcurrenciaOptimistaM5Repositorio.cs

CSHARP

```
public ConcurrenciaPropiedadesDistintasDto DemostrarCambiosEnPropiedadesDistintas()
{
    RestaurarEstadoInicial();
    using var scopeA = _scopeFactory.CreateScope();
    using var scopeB = _scopeFactory.CreateScope();
    var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();

    var ordenA = contextA.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);
    var ordenB = contextB.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);

    ordenA.Cliente = "Cliente actualizado por A";
    contextA.SaveChanges();
    ordenB.Estado = "EnProceso";
    contextB.SaveChanges();

    using var scopeVerificacion = _scopeFactory.CreateScope();
    var contextVerificacion = scopeVerificacion.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var resultado = contextVerificacion.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.NumeroOrden == NumeroOrden)
        .Select(o => new { o.Cliente, o.Estado })
        .Single();

    return new ConcurrenciaPropiedadesDistintasDto(
        ClienteInicial, EstadoInicial, resultado.Cliente, resultado.Estado,
        resultado.Cliente == ordenA.Cliente && resultado.Estado == ordenB.Estado,
        SqlCommandCounterInterceptor.Instance.SnapshotCommands());
}
```

Explicación línea a línea — ConcurrenciaOptimistaM5Repositorio.cs

Línea 1	<code>public ConcurrenciaPropiedadesDistintasDto DemostrarCambiosEnPropiedadesDistintas()</code> → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 2	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 3	<code>RestaurarEstadoInicial();</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 4	<code>using var scopeA = _scopeFactory.CreateScope();</code> → Crea un ámbito de DI independiente; los servicios Scoped, incluido <code>AceriaDbContext</code> , no se comparten con otros actores.
Línea 5	<code>using var scopeB = _scopeFactory.CreateScope();</code> → Crea un ámbito de DI independiente; los servicios Scoped, incluido <code>AceriaDbContext</code> , no se comparten con otros actores.
Línea 6	<code>var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve el <code>DbContext</code> del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.
Línea 7	<code>var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve el <code>DbContext</code> del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.
Línea 9	<code>var ordenA = contextA.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 10	<code>var ordenB = contextB.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 12	<code>ordenA.Cliente = "Cliente actualizado por A";</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 13	<code>contextA.SaveChanges();</code> → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 14	<code>ordenB.Estado = "EnProceso";</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 15	<code>contextB.SaveChanges();</code> → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.
Línea 17	<code>using var scopeVerificacion = _scopeFactory.CreateScope();</code> → Crea un ámbito de DI independiente; los servicios Scoped, incluido <code>AceriaDbContext</code> , no se comparten con otros actores.
Línea 18	<code>var contextVerificacion = scopeVerificacion.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve el <code>DbContext</code> del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.
Línea 19	<code>var resultado = contextVerificacion.OrdenesFabricacion</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 20	<code>.AsNoTracking()</code> → Ejecuta una lectura sin dejar entidades en el <code>ChangeTracker</code> .
Línea 21	<code>.Where(o => o.NumeroOrden == NumeroOrden)</code> → Añade el predicado a la consulta; EF Core intentará traducirlo al proveedor antes de materializar.
Línea 22	<code>.Select(o => new { o.Cliente, o.Estado })</code> → Proyecta únicamente la forma de datos necesaria para el resultado.
Línea 23	<code>.Single();</code> → Materializa o agrega la consulta en este punto, provocando la ejecución contra el proveedor cuando procede.
Línea 25	<code>return new ConcurrenciaPropiedadesDistintasDto(</code> → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.
Línea 26	<code>ClienteInicial, EstadoInicial, resultado.Cliente, resultado.Estado,</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 27	<code>resultado.Cliente == ordenA.Cliente && resultado.Estado == ordenB.Estado,</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 28	<code>SqlCommandCounterInterceptor.Instance.SnapshotCommands();</code> → Recupera los comandos SQL realmente interceptados durante la operación.
Línea 29	<code>}</code> → Delimita el bloque sintáctico correspondiente.

Paso 5: Compilar y ejecutar

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release --no-build
```

POWERSHELL

La ejecución debe finalizar con `5.1 OK`. Si una comprobación interna falla, la aplicación lanza una excepción y el punto no se considera correctamente reproducido.

Diagnóstico técnico

La práctica se considera correcta cuando se observan estas condiciones:

- Dos `DbContext` independientes cargan la misma orden antes de guardar.
- La misma propiedad permite observar una actualización perdida sin token.
- Propiedades distintas se conservan cuando EF Core genera `UPDATE` de las propiedades modificadas.
- La verificación final usa un tercer contexto `AsNoTracking`.
- El SQL mostrado procede del interceptor real.

Reto resuelto y ampliación

Reto: Añade una tercera escritura concurrente sobre Cliente y comprueba, mediante el SQL capturado, qué valor queda finalmente sin token.

Para resolverlo, conserva la misma regla del módulo: primero define qué resultado funcional debe mantenerse; después captura SQL, número de comandos, tracking, historial de migraciones o estado transaccional según corresponda. No uses una medición temporal aislada como sustituto de la evidencia técnica.

Errores comunes

- Usar el mismo DbContext para ambos actores y creer que se reproduce concurrencia.
- Modificar propiedades distintas y afirmar que necesariamente existe actualización perdida.
- Leer el resultado final desde un contexto con estado antiguo en el ChangeTracker.
- Inventar un SQL esperado en vez de observar el comando ejecutado.

Cierre acumulativo

1. Ejecuta `dotnet build` en la solución del punto.
2. Ejecuta la aplicación y comprueba el marcador `5.1 OK`.
3. Conserva las migraciones existentes; solo 5.2 añade el cambio de modelo oficial de M5.
4. Comprueba que Application continúa sin depender de Entity Framework Core ni de Infrastructure.
5. Antes de avanzar, relaciona el resultado con el SQL, la transacción, el historial o la prueba que lo demuestra.

Resultado esperado

Al finalizar el punto 5.1, AceriaData debe compilar, ejecutarse sobre SQL Server LocalDB y demostrar de forma observable el comportamiento descrito. El marcador final es:

5.1 OK

TEXT

Conexión con el siguiente punto

El siguiente estado es 5.2 y parte directamente de este proyecto completo.

Punto 5.2 — Configuración de tokens de concurrencia

Contexto del proyecto

Este punto continúa directamente `M05/PROYECTO/5.1`. La carpeta `M05/PROYECTO/5.2` contiene la solución completa `AceriaData.sln` y conserva las capas y migraciones existentes. Este punto introduce la migración real `M5_5_2_ConcurrencyTokens`.

Objetivo práctico

Configurar rowversion y un token de propiedad, generar la migración real y demostrar el conflicto sobre SQL Server LocalDB.

Paso 1: Abrir el estado completo del punto

```
cd M05/PROYECTO/5.2
dotnet restore AceriaData.sln
```

POWERSHELL

La restauración se ejecuta sobre la solución local del punto. No es necesario copiar archivos desde otros puntos: la carpeta ya contiene el estado acumulativo completo.

Paso 2: Identificar los archivos principales

En este punto se trabajan principalmente estos archivos:

- `src/AceriaData.Infrastructure/Persistence/Configurations/ConcurrencyTokensConfiguration.cs`
- `src/AceriaData.Infrastructure/Repositories/TokensConcurrenciaM5Repositorio.cs`

Los archivos se estudian dentro de su capa real. Application conserva sus abstracciones y casos de uso; Infrastructure contiene EF Core y acceso a proveedor; Console compone y ejecuta; los tests se ubican fuera de las capas de producción cuando aparecen en 5.9 y 5.11.

Paso 3: Implementar y estudiar `ConcurrencyTokensConfiguration.cs`

CSHARP

```
public sealed class OrdenFabricacionConcurrencyConfiguration : IEntityTypeConfiguration<OrdenFabricacion>
{
    public void Configure(EntityTypeBuilder<OrdenFabricacion> b) =>
        b.Property(x => x.RowVersion).IsRowVersion();
}

public sealed class PlanchaAceroConcurrencyConfiguration : IEntityTypeConfiguration<PlanchaAcero>
{
    public void Configure(EntityTypeBuilder<PlanchaAcero> b) =>
        b.Property(x => x.RowVersion).IsRowVersion();
}

public sealed class AleacionConcurrencyConfiguration : IEntityTypeConfiguration<Aleacion>
{
    public void Configure(EntityTypeBuilder<Aleacion> b) =>
        b.Property(x => x.RowVersion).IsRowVersion();
}

public sealed class DetalleOrdenConcurrencyConfiguration : IEntityTypeConfiguration<DetalleOrden>
{
    public void Configure(EntityTypeBuilder<DetalleOrden> b)
    {
        b.Property(x => x.EstadoDetalle)
            .IsRequired()
            .HasMaxLength(50)
            .HasDefaultValue("Pendiente")
            .IsConcurrencyToken();
    }
}
```

Explicación línea a línea — `ConcurrencyTokensConfiguration.cs`

Línea 1	<code>public sealed class OrdenFabricacionConcurrencyConfiguration :</code> <code>IEntityTypeConfiguration<OrdenFabricacion></code> → Declara la clase que encapsula la responsabilidad mostrada en este ejemplo.
Línea 2	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 3	<code>public void Configure(EntityTypeBuilder<OrdenFabricacion> b) =></code> → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 4	<code>b.Property(x => x.RowVersion).IsRowVersion();</code> → Configura la propiedad como token rowversion generado por SQL Server y usado por EF Core para concurrencia optimista.
Línea 5	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 7	<code>public sealed class PlanchaAceroConcurrencyConfiguration :</code> <code>IEntityTypeConfiguration<PlanchaAcero></code> → Declara la clase que encapsula la responsabilidad mostrada en este ejemplo.
Línea 8	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 9	<code>public void Configure(EntityTypeBuilder<PlanchaAcero> b) =></code> → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 10	<code>b.Property(x => x.RowVersion).IsRowVersion();</code> → Configura la propiedad como token rowversion generado por SQL Server y usado por EF Core para concurrencia optimista.

Línea 11	} → Delimita el bloque sintáctico correspondiente.
Línea 13	public sealed class AleacionConcurrencyConfiguration : IEntityTypeConfiguration<Aleacion> → Declara la clase que encapsula la responsabilidad mostrada en este ejemplo.
Línea 14	{ → Delimita el bloque sintáctico correspondiente.
Línea 15	public void Configure(EntityTypeBuilder<Aleacion> b) => → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 16	b.Property(x => x.RowVersion).IsRowVersion(); → Configura la propiedad como token rowversion generado por SQL Server y usado por EF Core para concurrencia optimista.
Línea 17	} → Delimita el bloque sintáctico correspondiente.
Línea 19	public sealed class DetalleOrdenConcurrencyConfiguration : IEntityTypeConfiguration<DetalleOrden> → Declara la clase que encapsula la responsabilidad mostrada en este ejemplo.
Línea 20	{ → Delimita el bloque sintáctico correspondiente.
Línea 21	public void Configure(EntityTypeBuilder<DetalleOrden> b) → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 22	{ → Delimita el bloque sintáctico correspondiente.
Línea 23	b.Property(x => x.EstadoDetalle) → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 24	.IsRequired() → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 25	.HasMaxLength(50) → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 26	.HasDefaultValue("Pendiente") → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 27	.IsConcurrencyToken(); → Marca la propiedad existente como token de concurrencia para que su valor original participe en escrituras.
Línea 28	} → Delimita el bloque sintáctico correspondiente.
Línea 29	} → Delimita el bloque sintáctico correspondiente.

Paso 4: Implementar y estudiar TokensConcurrenciaM5Repositorio.cs

```

public RowVersionM5Dto DemostrarRowVersion()
{
    using var scopeA = _scopeFactory.CreateScope();
    using var scopeB = _scopeFactory.CreateScope();
    var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();

    var ordenA = contextA.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);
    var ordenB = contextB.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);
    var inicialA = Convert.ToHexString(ordenA.RowVersion);
    var inicialB = Convert.ToHexString(ordenB.RowVersion);

    SqlCommandCounterInterceptor.Instance.Reset();
    ordenA.Cliente = "Cliente 5.2 A";
    contextA.SaveChanges();
    var despuesA = Convert.ToHexString(ordenA.RowVersion);

    ordenB.Estado = "EnProceso";
}

```

CSHARP

```

var conflicto = false;
try
{
    contextB.SaveChanges();
}
catch (DbUpdateConcurrencyException)
{
    conflicto = true;
}

return new RowVersionM5Dto(
    inicialA, inicialB, despuesA, conflicto,
    SqlCommandCounterInterceptor.Instance.SnapshotCommands());
}

```

Explicación línea a línea — TokensConcurrenciaM5Repositorio.cs

Línea 1	public RowVersionM5Dto DemostrarRowVersion() → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 2	{ → Delimita el bloque sintáctico correspondiente.
Línea 3	using var scopeA = _scopeFactory.CreateScope(); → Crea un ámbito de DI independiente; los servicios Scoped, incluido AeriaDbContext, no se comparten con otros actores.
Línea 4	using var scopeB = _scopeFactory.CreateScope(); → Crea un ámbito de DI independiente; los servicios Scoped, incluido AeriaDbContext, no se comparten con otros actores.
Línea 5	var contextA = scopeA.ServiceProvider.GetRequiredService<AeriaDbContext>(); → Resuelve el DbContext del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.
Línea 6	var contextB = scopeB.ServiceProvider.GetRequiredService<AeriaDbContext>(); → Resuelve el DbContext del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.
Línea 8	var ordenA = contextA.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden); → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 9	var ordenB = contextB.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden); → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 10	var inicialA = Convert.ToString(ordenA.RowVersion); → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 11	var inicialB = Convert.ToString(ordenB.RowVersion); → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 13	SqlCommandCounterInterceptor.Instance.Reset(); → Reinicia el contador para que solo se midan los comandos de este escenario.
Línea 14	ordenA.Cliente = "Cliente 5.2 A"; → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 15	contextA.SaveChanges(); → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.
Línea 16	var despuesA = Convert.ToString(ordenA.RowVersion); → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 18	ordenB.Estado = "EnProceso"; → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 19	<code>var conflicto = false;</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 20	<code>try</code> → Abre el bloque en el que se espera que pueda producirse la excepción que se quiere estudiar.
Línea 21	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 22	<code>contextB.SaveChanges();</code> → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.
Línea 23	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 24	<code>catch (DbUpdateConcurrencyException)</code> → Captura la excepción específica para aplicar la política de resolución prevista.
Línea 25	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 26	<code>conflicto = true;</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 27	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 29	<code>return new RowVersionM5Dto(</code> → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.
Línea 30	<code>inicialA, inicialB, despuesA, conflicto,</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 31	<code>SqlCommandCounterInterceptor.Instance.SnapshotCommands();</code> → Recupera los comandos SQL realmente interceptados durante la operación.
Línea 32	<code>}</code> → Delimita el bloque sintáctico correspondiente.

Paso 5: Implementar y estudiar `TokensConcurrenciaM5Repositorio.cs`

```

public IndiceRowVersionM5Dto ComprobarIndiceRowVersion()
{
    using var scope = _scopeFactory.CreateScope();
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var connection = context.Database.GetDbConnection();
    if (connection.State != ConnectionState.Open)
        connection.Open();

    using var command = connection.CreateCommand();
    command.CommandText = """
        SELECT COUNT(*)
        FROM sys.indexes AS i
        INNER JOIN sys.index_columns AS ic
            ON i.object_id = ic.object_id AND i.index_id = ic.index_id
        INNER JOIN sys.columns AS c
            ON ic.object_id = c.object_id AND ic.column_id = c.column_id
        WHERE i.object_id = OBJECT_ID(N'dbo.OrdenesFabricacion')
            AND c.name = N'RowVersion';
        """;

    return new IndiceRowVersionM5Dto(Convert.ToInt32(command.ExecuteScalar()) > 0);
}

```

CSHARP

Explicación línea a línea — `TokensConcurrenciaM5Repositorio.cs`

Línea 1	<code>public IndiceRowVersionM5Dto ComprobarIndiceRowVersion()</code> → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 2	<code>{</code> → Delimita el bloque sintáctico correspondiente.

Línea 3	<code>using var scope = _scopeFactory.CreateScope();</code> → Crea un ámbito de DI independiente; los servicios Scoped, incluido <code>AceriaDbContext</code> , no se comparten con otros actores.
Línea 4	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve el <code>DbContext</code> del ámbito y garantiza que la configuración de <code>Infrastructure</code> sea la misma que en la aplicación.
Línea 5	<code>var connection = context.Database.GetDbConnection();</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 6	<code>if (connection.State != ConnectionState.Open)</code> → Evalúa una condición necesaria y detiene o desvía el flujo cuando no se cumple.
Línea 7	<code>connection.Open();</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 9	<code>using var command = connection.CreateCommand();</code> → Importa el espacio de nombres necesario para los tipos y extensiones usados por el archivo, o declara un recurso con liberación automática cuando se usa <code>using var</code> .
Línea 10	<code>command.CommandText = ""</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 11	<code>SELECT COUNT(*)</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 12	<code>FROM sys.indexes AS i</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 13	<code>INNER JOIN sys.index_columns AS ic</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 14	<code>ON i.object_id = ic.object_id AND i.index_id = ic.index_id</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 15	<code>INNER JOIN sys.columns AS c</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 16	<code>ON ic.object_id = c.object_id AND ic.column_id = c.column_id</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 17	<code>WHERE i.object_id = OBJECT_ID(N'dbo.OrdenesFabricacion')</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 18	<code>AND c.name = N'RowVersion';</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 19	<code>"";</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 21	<code>return new IndiceRowVersionM5Dto(Convert.ToInt32(command.ExecuteScalar()) > 0);</code> → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.
Línea 22	<code>}</code> → Delimita el bloque sintáctico correspondiente.

Paso 6: Compilar y ejecutar

POWERSHELL

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console --configuration Release
dotnet ef database update --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console --configuration Release
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release --no-build
```

La ejecución debe finalizar con `5.2 OK`. Si una comprobación interna falla, la aplicación lanza una excepción y el punto no se considera correctamente reproducido.

Diagnóstico técnico

La práctica se considera correcta cuando se observan estas condiciones:

- RowVersion se configura explícitamente con `IsRowVersion()`.
- EstadoDetalle se configura como `IsConcurrencyToken()`.
- La migración `M5_5_2_ConcurrencyTokens` existe y se aplica.
- Dos contextos producen `DbUpdateConcurrencyException` cuando cambia el token.
- Se comprueba directamente que RowVersion no obtiene un índice automático.

Reto resuelto y ampliación

Reto: Inspecciona el UPDATE capturado y localiza el token original dentro del predicado de concurrencia.

Para resolverlo, conserva la misma regla del módulo: primero define qué resultado funcional debe mantenerse; después captura SQL, número de comandos, tracking, historial de migraciones o estado transaccional según corresponda. No uses una medición temporal aislada como sustituto de la evidencia técnica.

Errores comunes

- Confiar en el nombre RowVersion sin configurarlo.
- Suponer que rowversion crea automáticamente un índice.
- Probar semántica de rowversion de SQL Server con SQLite.
- Modificar a mano el snapshot o la migración para forzar el resultado.

Cierre acumulativo

1. Ejecuta `dotnet build` en la solución del punto.
2. Ejecuta la aplicación y comprueba el marcador `5.2 OK`.
3. Conserva las migraciones existentes; solo 5.2 añade el cambio de modelo oficial de M5.
4. Comprueba que Application continúa sin depender de Entity Framework Core ni de Infrastructure.
5. Antes de avanzar, relaciona el resultado con el SQL, la transacción, el historial o la prueba que lo demuestra.

Resultado esperado

Al finalizar el punto 5.2, AceriaData debe compilar, ejecutarse sobre SQL Server LocalDB y demostrar de forma observable el comportamiento descrito. El marcador final es:

5.2 OK

TEXT

Conexión con el siguiente punto

El siguiente estado es 5.3 y parte directamente de este proyecto completo.

Punto 5.3 — Resolución de conflictos de concurrencia

Contexto del proyecto

Este punto continúa directamente `M05/PROYECTO/5.2`. La carpeta `M05/PROYECTO/5.3` contiene la solución completa `AceriaData.sln` y conserva las capas y migraciones existentes. El modelo oficial no cambia en este punto; se conserva la última migración real ya existente.

Objetivo práctico

Resolver `DbUpdateConcurrencyException` con políticas explícitas: cliente gana, base gana, merge, notificación, fila eliminada y reintento acotado.

Paso 1: Abrir el estado completo del punto

```
cd M05/PROYECTO/5.3
dotnet restore AceriaData.sln
```

POWERSHELL

La restauración se ejecuta sobre la solución local del punto. No es necesario copiar archivos desde otros puntos: la carpeta ya contiene el estado acumulativo completo.

Paso 2: Identificar los archivos principales

En este punto se trabajan principalmente estos archivos:

- `src/AceriaData.Infrastructure/Repositories/ResolucionConflictosM5Repositorio.cs`

Los archivos se estudian dentro de su capa real. Application conserva sus abstracciones y casos de uso; Infrastructure contiene EF Core y acceso a proveedor; Console compone y ejecuta; los tests se ubican fuera de las capas de producción cuando aparecen en 5.9 y 5.11.

Paso 3: Implementar y estudiar `ResolucionConflictosM5Repositorio.cs`

```
public ResolucionConflictoM5Dto ClienteGana()
{
    RestaurarOrden();
    using var scopeA = _scopeFactory.CreateScope();
    using var scopeB = _scopeFactory.CreateScope();
    var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var a = Cargar(contextA);
    var b = Cargar(contextB);

    a.Cliente = "Cliente A - cliente gana";
    contextA.SaveChanges();

    b.Cliente = "Cliente B - cliente gana";
    var conflicto = false;
    var intentos = 1;
    var valores = new List<ValorConflictoM5Dto>();

    try
    {
        contextB.SaveChanges();
    }
    catch (DbUpdateConcurrencyException ex)
    {
        conflicto = true;
        var entry = ex.Entries.Single();
        var db = entry.GetDatabaseValues()
            ?? throw new InvalidOperationException("La orden desapareció durante ClienteGana.");
        valores = CrearValores(entry, db);
        entry.OriginalValues.SetValues(db);
        intentos++;
        contextB.SaveChanges();
    }

    var final = LeerFinal();
    return Resultado("Cliente gana", conflicto, final, intentos, valores);
}
```

CSHARP

Explicación línea a línea — `ResolucionConflictosM5Repositorio.cs`

Línea 1	<code>public ResolucionConflictoM5Dto ClienteGana()</code> → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 2	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 3	<code>RestaurarOrden();</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 4	<code>using var scopeA = _scopeFactory.CreateScope();</code> → Crea un ámbito de DI independiente; los servicios Scoped, incluido <code>AceriaDbContext</code> , no se comparten con otros actores.

Línea 5	<code>using var scopeB = _scopeFactory.CreateScope();</code> → Crea un ámbito de DI independiente; los servicios Scoped, incluido <code>AceriaDbContext</code> , no se comparten con otros actores.
Línea 6	<code>var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve el <code>DbContext</code> del ámbito y garantiza que la configuración de <code>Infrastructure</code> sea la misma que en la aplicación.
Línea 7	<code>var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve el <code>DbContext</code> del ámbito y garantiza que la configuración de <code>Infrastructure</code> sea la misma que en la aplicación.
Línea 8	<code>var a = Cargar(contextA);</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 9	<code>var b = Cargar(contextB);</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 11	<code>a.Cliente = "Cliente A - cliente gana";</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 12	<code>contextA.SaveChanges();</code> → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.
Línea 14	<code>b.Cliente = "Cliente B - cliente gana";</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 15	<code>var conflicto = false;</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 16	<code>var intentos = 1;</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 17	<code>var valores = new List<ValorConflictoM5Dto>();</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 19	<code>try</code> → Abre el bloque en el que se espera que pueda producirse la excepción que se quiere estudiar.
Línea 20	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 21	<code>contextB.SaveChanges();</code> → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.
Línea 22	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 23	<code>catch (DbUpdateConcurrencyException ex)</code> → Captura la excepción específica para aplicar la política de resolución prevista.
Línea 24	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 25	<code>conflicto = true;</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 26	<code>var entry = ex.Entries.Single();</code> → Materializa o agrega la consulta en este punto, provocando la ejecución contra el proveedor cuando procede.
Línea 27	<code>var db = entry.GetDatabaseValues();</code> → Obtiene de la base el estado actual de la fila tras detectar el conflicto.
Línea 28	<code>?? throw new InvalidOperationException("La orden desapareció durante ClienteGana.");</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 29	<code>valores = CrearValores(entry, db);</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 30	<code>entry.OriginalValues.SetValues(db);</code> → Actualiza los valores originales del entry para reintentar contra la versión recién leída.
Línea 31	<code>intentos++;</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 32	<code>contextB.SaveChanges();</code> → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.
Línea 33	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 35	<code>var final = LeerFinal();</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 36	<code>return Resultado("Cliente gana", conflicto, final, intentos, valores);</code> → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.
Línea 37	<code>}</code> → Delimita el bloque sintáctico correspondiente.

Paso 4: Implementar y estudiar `ResolucionConflictosM5Repositorio.cs`

```

public ResolucionConflictoM5Dto ResolucionPersonalizada()
{
    RestaurarOrden();
    using var scopeA = _scopeFactory.CreateScope();
    using var scopeB = _scopeFactory.CreateScope();
    var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var a = Cargar(contextA);
    var b = Cargar(contextB);

    a.Cliente = "Cliente A - merge";
    a.Estado = "EnProceso A";
    contextA.SaveChanges();

    b.Cliente = "Cliente B - merge";
    b.Estado = "Completada B";
    var intentos = 1;

    try
    {
        contextB.SaveChanges();
    }
    catch (DbUpdateConcurrencyException ex)
    {
        var entry = ex.Entries.Single();
        var db = entry.GetDatabaseValues()
            ?? throw new InvalidOperationException("La orden desapareció durante la fusión.");
        entry.OriginalValues.SetValues(db);
        entry.CurrentValues[nameof(OrdenFabricacion.Estado)] = db[nameof(OrdenFabricacion.Estado)];
        intentos++;
        contextB.SaveChanges();
    }

    return Resultado("Resolución personalizada", true, LeerFinal(), intentos, []);
}

```

CSHARP

Explicación línea a línea — `ResolucionConflictosM5Repositorio.cs`

Línea 1	<code>public ResolucionConflictoM5Dto ResolucionPersonalizada()</code> → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 2	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 3	<code>RestaurarOrden();</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 4	<code>using var scopeA = _scopeFactory.CreateScope();</code> → Crea un ámbito de DI independiente; los servicios Scoped, incluido <code>AceriaDbContext</code> , no se comparten con otros actores.

Línea 5	<code>using var scopeB = _scopeFactory.CreateScope();</code> → Crea un ámbito de DI independiente; los servicios Scoped, incluido <code>AceriaDbContext</code> , no se comparten con otros actores.
Línea 6	<code>var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve el <code>DbContext</code> del ámbito y garantiza que la configuración de <code>Infrastructure</code> sea la misma que en la aplicación.
Línea 7	<code>var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve el <code>DbContext</code> del ámbito y garantiza que la configuración de <code>Infrastructure</code> sea la misma que en la aplicación.
Línea 8	<code>var a = Cargar(contextA);</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 9	<code>var b = Cargar(contextB);</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 11	<code>a.Cliente = "Cliente A - merge";</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 12	<code>a.Estado = "EnProceso A";</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 13	<code>contextA.SaveChanges();</code> → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.
Línea 15	<code>b.Cliente = "Cliente B - merge";</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 16	<code>b.Estado = "Completada B";</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 17	<code>var intentos = 1;</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 19	<code>try</code> → Abre el bloque en el que se espera que pueda producirse la excepción que se quiere estudiar.
Línea 20	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 21	<code>contextB.SaveChanges();</code> → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.
Línea 22	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 23	<code>catch (DbUpdateConcurrencyException ex)</code> → Captura la excepción específica para aplicar la política de resolución prevista.
Línea 24	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 25	<code>var entry = ex.Entries.Single();</code> → Materializa o agrega la consulta en este punto, provocando la ejecución contra el proveedor cuando procede.
Línea 26	<code>var db = entry.GetDatabaseValues();</code> → Obtiene de la base el estado actual de la fila tras detectar el conflicto.
Línea 27	<code>?? throw new InvalidOperationException("La orden desapareció durante la fusión.");</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 28	<code>entry.OriginalValues.SetValues(db);</code> → Actualiza los valores originales del <code>entry</code> para reintentar contra la versión recién leída.

Línea 29	<code>entry.CurrentValues[nameof(OrdenFabricacion.Estado)] = db[nameof(OrdenFabricacion.Estado)];</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 30	<code>intentos++;</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 31	<code>contextB.SaveChanges();</code> → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.
Línea 32	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 34	<code>return Resultado("Resolución personalizada", true, LeerFinal(), intentos, []);</code> → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.
Línea 35	<code>}</code> → Delimita el bloque sintáctico correspondiente.

Paso 5: Implementar y estudiar `ResolucionConflictosM5Repositorio.cs`

CSHARP

```
while (true)
{
    intentos++;
    try
    {
        contextB.SaveChanges();
        break;
    }
    catch (DbUpdateConcurrencyException ex) when (intentos < maxIntentos)
    {
        conflicto = true;
        var entry = ex.Entries.Single();
        var db = entry.GetDatabaseValues()
            ?? throw new InvalidOperationException("La fila ya no existe.");
        entry.OriginalValues.SetValues(db);
    }
}
```

Explicación línea a línea — `ResolucionConflictosM5Repositorio.cs`

Línea 1	<code>while (true)</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 2	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 3	<code>intentos++;</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 4	<code>try</code> → Abre el bloque en el que se espera que pueda producirse la excepción que se quiere estudiar.
Línea 5	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 6	<code>contextB.SaveChanges();</code> → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.
Línea 7	<code>break;</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 8	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 9	<code>catch (DbUpdateConcurrencyException ex) when (intentos < maxIntentos)</code> → Captura la excepción específica para aplicar la política de resolución prevista.
Línea 10	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 11	<code>conflicto = true;</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 12	<code>var entry = ex.Entries.Single();</code> → Materializa o agrega la consulta en este punto, provocando la ejecución contra el proveedor cuando procede.
Línea 13	<code>var db = entry.GetDatabaseValues();</code> → Obtiene de la base el estado actual de la fila tras detectar el conflicto.
Línea 14	<code>?? throw new InvalidOperationException("La fila ya no existe.");</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 15	<code>entry.OriginalValues.SetValues(db);</code> → Actualiza los valores originales del entry para reintentar contra la versión recién leída.
Línea 16	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 17	<code>}</code> → Delimita el bloque sintáctico correspondiente.

Paso 6: Compilar y ejecutar

POWERSHELL

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet ef migrations has-pending-model-changes --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console --configuration Release
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release --no-build
```

La ejecución debe finalizar con `5.3 OK`. Si una comprobación interna falla, la aplicación lanza una excepción y el punto no se considera correctamente reproducido.

Diagnóstico técnico

La práctica se considera correcta cuando se observan estas condiciones:

- `GetDatabaseValues` recupera el estado actual de SQL Server.
- `OriginalValues.SetValues` actualiza la versión contra la que se reintenta.
- `Reload` descarta cambios locales cuando la base gana.
- La resolución personalizada decide propiedad por propiedad.
- Los reintentos están limitados.

Reto resuelto y ampliación

Reto: Añade una política que preserve Cliente local, Estado de base y combine Observaciones solo cuando ambos lados no sean nulos.

Para resolverlo, conserva la misma regla del módulo: primero define qué resultado funcional debe mantenerse; después captura SQL, número de comandos, tracking, historial de migraciones o estado transaccional según corresponda. No uses una medición temporal aislada como sustituto de la evidencia técnica.

Errores comunes

- Reintentar indefinidamente.
- Confundir `CurrentValues`, `OriginalValues` y valores actuales de base.
- Sobrescribir automáticamente sin informar al usuario cuando el negocio requiere decisión.
- No tratar `GetDatabaseValues()=null` cuando la fila fue eliminada.

Cierre acumulativo

1. Ejecuta `dotnet build` en la solución del punto.
2. Ejecuta la aplicación y comprueba el marcador `5.3 OK`.
3. Conserva las migraciones existentes; solo 5.2 añade el cambio de modelo oficial de M5.
4. Comprueba que `Application` continúa sin depender de Entity Framework Core ni de Infrastructure.
5. Antes de avanzar, relaciona el resultado con el SQL, la transacción, el historial o la prueba que lo demuestra.

Resultado esperado

Al finalizar el punto 5.3, AceriaData debe compilar, ejecutarse sobre SQL Server LocalDB y demostrar de forma observable el comportamiento descrito. El marcador final es:

5.3 OK

TEXT

Conexión con el siguiente punto

El siguiente estado es 5.4 y parte directamente de este proyecto completo.

Punto 5.4 — Transacciones: SaveChanges y transacciones explícitas

Contexto del proyecto

Este punto continúa directamente M05/PROYECTO/5.3. La carpeta M05/PROYECTO/5.4 contiene la solución completa AceriaData.sln y conserva las capas y migraciones existentes. El modelo oficial no cambia en este punto; se conserva la última migración real ya existente.

Objetivo práctico

Demostrar atomicidad de SaveChanges, Commit, Rollback y savepoints reales con SQL Server y MARS desactivado.

Paso 1: Abrir el estado completo del punto

```
cd M05/PROYECTO/5.4
dotnet restore AceriaData.sln
```

POWERSHELL

La restauración se ejecuta sobre la solución local del punto. No es necesario copiar archivos desde otros puntos: la carpeta ya contiene el estado acumulativo completo.

Paso 2: Identificar los archivos principales

En este punto se trabajan principalmente estos archivos:

- src/AceriaData.Infrastructure/Repositories/TransaccionesM5Repositorio.cs
- src/AceriaData.Console/appsettings.json

Los archivos se estudian dentro de su capa real. Application conserva sus abstracciones y casos de uso; Infrastructure contiene EF Core y acceso a proveedor; Console compone y ejecuta; los tests se ubican fuera de las capas de producción cuando aparecen en 5.9 y 5.11.

Paso 3: Implementar y estudiar TransaccionesM5Repositorio.cs

```
public ResultadoTransaccionM5Dto DemostrarAtomicidadSaveChanges()
{
    const string numeroValido = "0F-M5-54-ATOMIC";
    Limpiar(numeroValido);
    using var scope = _scopeFactory.CreateScope();
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();

    context.OrdenesFabricacion.Add(CrearOrden(numeroValido));
    context.OrdenesFabricacion.Add(CrearOrden(NumeroExistente));

    var falloEsperado = false;
    try
    {
        context.SaveChanges();
    }
    catch (DbUpdateException)
    {
    }
```

CSHARP

```

        falloEsperado = true;
        context.ChangeTracker.Clear();
    }

    var existeValida = Existe(numeroValido);
    return new ResultadoTransaccionM5Dto(
        "Atomicidad de un único SaveChanges",
        falloEsperado && !existeValida,
        existeValida,
        Existe(NumeroExistente),
        Marshabilitado(context),
        SqlCommandCounterInterceptor.Instance.SnapshotCommands());
}

```

Explicación línea a línea — TransaccionesM5Repositorio.cs

Línea 1	<code>public ResultadoTransaccionM5Dto DemostrarAtomicidadSaveChanges()</code> → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 2	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 3	<code>const string numeroValido = "0F-M5-54-ATOMIC";</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 4	<code>Limpiar(numeroValido);</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 5	<code>using var scope = _scopeFactory.CreateScope();</code> → Crea un ámbito de DI independiente; los servicios Scoped, incluido <code>AceriaDbContext</code> , no se comparten con otros actores.
Línea 6	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve el <code>DbContext</code> del ámbito y garantiza que la configuración de Infraestructure sea la misma que en la aplicación.
Línea 8	<code>context.OrdenesFabricacion.Add(CrearOrden(numeroValido));</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 9	<code>context.OrdenesFabricacion.Add(CrearOrden(NumeroExistente));</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 11	<code>var falloEsperado = false;</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 12	<code>try</code> → Abre el bloque en el que se espera que pueda producirse la excepción que se quiere estudiar.
Línea 13	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 14	<code>context.SaveChanges();</code> → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.
Línea 15	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 16	<code>catch (DbUpdateException)</code> → Captura la excepción específica para aplicar la política de resolución prevista.
Línea 17	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 18	<code>falloEsperado = true;</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 19	<code>context.ChangeTracker.Clear();</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 20	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 22	<code>var existeValida = Existe(numeroValido);</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 23	<code>return new ResultadoTransaccionM5Dto(</code> → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.
Línea 24	<code>"Atomicidad de un único SaveChanges",</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 25	<code>falloEsperado && !existeValida,</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 26	<code>existeValida,</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 27	<code>Existe(NumeroExistente),</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 28	<code>MarsHabilitado(context),</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 29	<code>SqlCommandCounterInterceptor.Instance.SnapshotCommands();</code> → Recupera los comandos SQL realmente interceptados durante la operación.
Línea 30	<code>}</code> → Delimita el bloque sintáctico correspondiente.

Paso 4: Implementar y estudiar `TransaccionesM5Repositorio.cs`

CSHARP

```
public ResultadoTransaccionM5Dto DemostrarRollbackASavepoint()
{
    const string numero1 = "0F-M5-54-SP-1";
    const string numero2 = "0F-M5-54-SP-2";
    const string savepoint = "AntesSegundaOrden";
    Limpiar(numero1, numero2);

    using var scope = _scopeFactory.CreateScope();
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var mars = MarsHabilitado(context);
    if (mars)
        throw new InvalidOperationException("5.4 requiere MultipleActiveResultSets=false.");

    using (var transaction = context.Database.BeginTransaction())
    {
        context.OrdenesFabricacion.Add(CrearOrden(numero1));
        context.SaveChanges();
        transaction.CreateSavepoint(savepoint);

        context.OrdenesFabricacion.Add(CrearOrden(numero2));
        context.SaveChanges();
        transaction.RollbackToSavepoint(savepoint);

        context.ChangeTracker.Clear();
        transaction.Commit();
    }

    return new ResultadoTransaccionM5Dto(
        "Rollback a savepoint",
        Existe(numero1) && !Existe(numero2),
        Existe(numero1), Existe(numero2), mars,
        SqlCommandCounterInterceptor.Instance.SnapshotCommands());
}
```

Explicación línea a línea — `TransaccionesM5Repositorio.cs`

Línea 1	<code>public ResultadoTransaccionM5Dto DemostrarRollbackASavepoint()</code> → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 2	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 3	<code>const string numero1 = "0F-M5-54-SP-1";</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 4	<code>const string numero2 = "0F-M5-54-SP-2";</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 5	<code>const string savepoint = "AntesSegundaOrden";</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 6	<code>Limpiar(numero1, numero2);</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 8	<code>using var scope = _scopeFactory.CreateScope();</code> → Crea un ámbito de DI independiente; los servicios Scoped, incluido <code>AceriaDbContext</code> , no se comparten con otros actores.
Línea 9	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve el <code>DbContext</code> del ámbito y garantiza que la configuración de <code>Infrastructure</code> sea la misma que en la aplicación.
Línea 10	<code>var mars = MarsHabilitado(context);</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 11	<code>if (mars)</code> → Evalúa una condición necesaria y detiene o desvía el flujo cuando no se cumple.
Línea 12	<code>throw new InvalidOperationException("5.4 requiere MultipleActiveResultSets=false.");</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 14	<code>using (var transaction = context.Database.BeginTransaction())</code> → Abre una transacción explícita sobre la conexión del <code>DbContext</code> .
Línea 15	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 16	<code>context.OrdenesFabricacion.Add(CrearOrden(numero1));</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 17	<code>context.SaveChanges();</code> → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.
Línea 18	<code>transaction.CreateSavepoint(savepoint);</code> → Crea un punto intermedio al que puede volver la transacción sin descartar todo el trabajo previo.
Línea 20	<code>context.OrdenesFabricacion.Add(CrearOrden(numero2));</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 21	<code>context.SaveChanges();</code> → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.
Línea 22	<code>transaction.RollbackToSavepoint(savepoint);</code> → Revierte solo las operaciones posteriores al <code>savepoint</code> indicado.
Línea 24	<code>context.ChangeTracker.Clear();</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 25	<code>transaction.Commit();</code> → Marca/confirma la unidad transaccional cuando todas las operaciones requeridas han terminado correctamente.
Línea 26	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 28	<code>return new ResultadoTransaccionM5Dto(</code> → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.
Línea 29	<code>"Rollback a savepoint",</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 30	<code>Existe(numero1) && !Existe(numero2),</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 31	<code>Existe(numero1), Existe(numero2), mars, →</code> Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 32	<code>SqlCommandCounterInterceptor.Instance.SnapshotCommands()); →</code> Recupera los comandos SQL realmente interceptados durante la operación.
Línea 33	<code>}</code> → Delimita el bloque sintáctico correspondiente.

Paso 5: Implementar y estudiar `appsettings.json`

```
{
  "ConnectionStrings": {
    "AceriaDB": "Server=(localdb)\\
\\MSSQLLocalDB;Database=AceriaDB;Trusted_Connection=True;TrustServerCertificate=True;MultipleActiveResultSets=false;"
  }
}
```

JSON

Explicación línea a línea — `appsettings.json`

Línea 1	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 2	<code>"ConnectionStrings": {</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 3	<code>"AceriaDB": "Server=(localdb)\\ \\MSSQLLocalDB;Database=AceriaDB;Trusted_Connection=True;TrustServerCertificate=True;Multiple ActiveResultSets=false;"</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 4	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 5	<code>}</code> → Delimita el bloque sintáctico correspondiente.

Paso 6: Compilar y ejecutar

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet ef migrations has-pending-model-changes --project src/AceriaData.Infrastructure --startup-project src/
AceriaData.Console --configuration Release
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release --no-build
```

POWERSHELL

La ejecución debe finalizar con `5.4 OK`. Si una comprobación interna falla, la aplicación lanza una excepción y el punto no se considera correctamente reproducido.

Diagnóstico técnico

La práctica se considera correcta cuando se observan estas condiciones:

- Un único `SaveChanges` revierte todo el conjunto si una escritura falla.
- Una transacción explícita puede agrupar varias llamadas a `SaveChanges`.
- `Rollback` deshace una escritura ya enviada antes del fallo posterior.
- `RollbackToSavepoint` conserva lo anterior al savepoint y revierte lo posterior.
- `MultipleActiveResultSets=false` en el laboratorio de savepoints.

Reto resuelto y ampliación

Reto: Introduce un tercer `SaveChanges` después del savepoint y decide qué parte debe persistir tras un rollback parcial.

Para resolverlo, conserva la misma regla del módulo: primero define qué resultado funcional debe mantenerse; después captura SQL, número de comandos, tracking, historial de migraciones o estado transaccional según corresponda. No uses una medición temporal aislada como sustituto de la evidencia técnica.

Errores comunes

- Dar por hecho que Commit fallido siempre implica un estado conocido.
- Usar MARS y esperar savepoints automáticos de EF Core.
- Presentar RELEASE SAVEPOINT como operación necesaria de SQL Server.
- Mantener transacciones abiertas durante trabajo ajeno a la base.

Cierre acumulativo

1. Ejecuta `dotnet build` en la solución del punto.
2. Ejecuta la aplicación y comprueba el marcador `5.4 OK`.
3. Conserva las migraciones existentes; solo 5.2 añade el cambio de modelo oficial de M5.
4. Comprueba que Application continúa sin depender de Entity Framework Core ni de Infrastructure.
5. Antes de avanzar, relaciona el resultado con el SQL, la transacción, el historial o la prueba que lo demuestra.

Resultado esperado

Al finalizar el punto 5.4, AceriaData debe compilar, ejecutarse sobre SQL Server LocalDB y demostrar de forma observable el comportamiento descrito. El marcador final es:

5.4 OK

TEXT

Conexión con el siguiente punto

El siguiente estado es 5.5 y parte directamente de este proyecto completo.

Punto 5.5 — Transacciones ambientales y buenas prácticas

Contexto del proyecto

Este punto continúa directamente `M05/PROYECTO/5.4`. La carpeta `M05/PROYECTO/5.5` contiene la solución completa `AceriaData.sln` y conserva las capas y migraciones existentes. El modelo oficial no cambia en este punto; se conserva la última migración real ya existente.

Objetivo práctico

Usar `TransactionScope` con `async`, varios `DbContext` sobre una conexión compartida, opciones de scope y un efecto externo no transaccional.

Paso 1: Abrir el estado completo del punto

```
cd M05/PROYECTO/5.5
dotnet restore AceriaData.sln
```

POWERSHELL

La restauración se ejecuta sobre la solución local del punto. No es necesario copiar archivos desde otros puntos: la carpeta ya contiene el estado acumulativo completo.

Paso 2: Identificar los archivos principales

En este punto se trabajan principalmente estos archivos:

- `src/AceriaData.Infrastructure/Repositories/TransaccionesAmbientalesM5Repositorio.cs`

Los archivos se estudian dentro de su capa real. Application conserva sus abstracciones y casos de uso; Infrastructure contiene EF Core y acceso a proveedor; Console compone y ejecuta; los tests se ubican fuera de las capas de producción cuando aparecen en 5.9 y 5.11.

Paso 3: Implementar y estudiar TransaccionesAmbientalesM5Repositorio.cs

CSHARP

```
public async Task<TransaccionAmbientaIM5Dto> DemostrarDosContextosAsync()
{
    var options = new TransactionOptions
    {
        IsolationLevel = IsolationLevel.ReadCommitted,
        Timeout = TimeSpan.FromSeconds(30)
    };

    bool ambienteActivo;
    bool flujoAsync;
    bool promocion;
    string aislamiento;
    int ordenId;

    using (var scope = new TransactionScope(
        TransactionScopeOption.Required,
        options,
        TransactionScopeAsyncFlowOption.Enabled))
    {
        await using var connection = new SqlConnection(ObtenerConnectionString());
        await connection.OpenAsync();
        ambienteActivo = Transaction.Current is not null;
        aislamiento = Transaction.Current?.IsolationLevel.ToString() ?? "<none>";

        await using (var contextOrden = CrearContexto(connection))
        {
            var orden = CrearOrden("OF-M5-55-AMBIENT");
            contextOrden.OrdenesFabricacion.Add(orden);
            await contextOrden.SaveChangesAsync();
            ordenId = orden.Id;
        }

        await Task.Yield();
        flujoAsync = Transaction.Current is not null;

        await using (var contextPlancha = CrearContexto(connection))
        {
            contextPlancha.PlanchasAcero.Add(new PlanchaAcero
            {
                OrdenId = ordenId,
                Espesor = 10, Ancho = 1000, Largo = 2000, Peso = 150m
            });
            await contextPlancha.SaveChangesAsync();
        }

        promocion = Transaction.Current?.TransactionInformation.DistributedIdentifier != Guid.Empty;
        scope.Complete();
    }

    return new TransaccionAmbientaIM5Dto(
        "Dos DbContext con una conexión compartida",
        ExisteOrdenYPlancha("OF-M5-55-AMBIENT"),
        ambienteActivo, flujoAsync, promocion, aislamiento,
        SqlCommandCounterInterceptor.Instance.SnapshotCommands());
}
```

Explicación línea a línea — TransaccionesAmbientalesM5Repositorio.cs

Línea 1	public async Task<TransaccionAmbientaIM5Dto> DemostrarDosContextosAsync() → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 2	{ → Delimita el bloque sintáctico correspondiente.
Línea 3	var options = new TransactionOptions → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 4	{ → Delimita el bloque sintáctico correspondiente.
Línea 5	IsolationLevel = IsolationLevel.ReadCommitted, → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 6	Timeout = TimeSpan.FromSeconds(30) → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 7	<code>};</code> → Delimita el bloque sintáctico correspondiente.
Línea 9	<code>bool ambienteActivo;</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 10	<code>bool flujoAsync;</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 11	<code>bool promocion;</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 12	<code>string aislamiento;</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 13	<code>int ordenId;</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 15	<code>using (var scope = new TransactionScope(</code> → Crea o participa en una transacción ambiental según la opción indicada.
Línea 16	<code>TransactionScopeOption.Required,</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 17	<code>options,</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 18	<code>TransactionScopeAsyncFlowOption.Enabled))</code> → Permite que <code>Transaction.Current</code> fluya correctamente a través de <code>await</code> .
Línea 19	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 20	<code>await using var connection = new SqlConnection(ObtenerConnectionString());</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 21	<code>await connection.OpenAsync();</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 22	<code>ambienteActivo = Transaction.Current is not null;</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 23	<code>aislamiento = Transaction.Current?.IsolationLevel.ToString() ?? "<none>";</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 25	<code>await using (var contextOrden = CrearContexto(connection))</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 26	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 27	<code>var orden = CrearOrden("OF-M5-55-AMBIENT");</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 28	<code>contextOrden.OrdenesFabricacion.Add(orden);</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 29	<code>await contextOrden.SaveChangesAsync();</code> → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.
Línea 30	<code>ordenId = orden.Id;</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 31	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 33	<code>await Task.Yield();</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 34	<code>flujoAsync = Transaction.Current is not null;</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 36	<code>await using (var contextPlancha = CrearContexto(connection))</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 37	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 38	<code>contextPlancha.PlanchasAcero.Add(new PlanchaAcero</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 39	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 40	<code>OrdenId = ordenId,</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 41	<code>Espesor = 10, Ancho = 1000, Largo = 2000, Peso = 150m</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 42	<code>});</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 43	<code>await contextPlancha.SaveChangesAsync();</code> → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.
Línea 44	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 46	<code>promocion = Transaction.Current?.TransactionInformation.DistributedIdentifier != Guid.Empty;</code> → Comprueba si la transacción fue promocionada a una transacción distribuida.
Línea 47	<code>scope.Complete();</code> → Marca/confirma la unidad transaccional cuando todas las operaciones requeridas han terminado correctamente.
Línea 48	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 50	<code>return new TransaccionAmbientaM5Dto(</code> → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.
Línea 51	<code>"Dos DbContext con una conexión compartida",</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 52	<code>ExisteOrdenYPlancha("0F-M5-55-AMBIENT"),</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 53	<code>ambienteActivo, flujoAsync, promocion, aislamiento,</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 54	<code>SqlCommandCounterInterceptor.Instance.SnapshotCommands());</code> → Recupera los comandos SQL realmente interceptados durante la operación.
Línea 55	<code>}</code> → Delimita el bloque sintáctico correspondiente.

Paso 4: Implementar y estudiar `TransaccionesAmbientalesM5Repositorio.cs`

CSHARP

```
public OpcionesTransactionScopeM5Dto DemostrarOpcionesDeScope()
{
    string outerId;
    bool requiredMisma;
    bool requiresNewOtra;
    bool suppressSinAmbiente;
    string aislamiento;

    using (var outer = new TransactionScope())
    {
        var outerTransaction = Transaction.Current
```

```

        ?? throw new InvalidOperationException("No se creó la transacción ambiental.");
        outerId = outerTransaction.TransactionInformation.LocalIdentifier;
        aislamiento = outerTransaction.IsolationLevel.ToString();

        using (var required = new TransactionScope(TransactionScopeOption.Required))
        {
            requiredMisma = Transaction.Current?.TransactionInformation.LocalIdentifier == outerId;
            required.Complete();
        }

        using (var requiresNew = new TransactionScope(TransactionScopeOption.RequiresNew))
        {
            requiresNewOtra = Transaction.Current?.TransactionInformation.LocalIdentifier != outerId;
            requiresNew.Complete();
        }

        using (var suppressed = new TransactionScope(TransactionScopeOption.Suppress))
        {
            suppressSinAmbiente = Transaction.Current is null;
            suppressed.Complete();
        }
        outer.Complete();
    }

    return new OpcionesTransactionScopeM5Dto(
        requiredMisma, requiresNewOtra, suppressSinAmbiente,
        aislamiento, TransactionManager.DefaultTimeout.TotalSeconds);
}

```

Explicación línea a línea — TransaccionesAmbientalesM5Repositorio.cs

Línea 1	<code>public OpcionesTransactionScopeM5Dto DemostrarOpcionesDeScope()</code> → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 2	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 3	<code>string outerId;</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 4	<code>bool requiredMisma;</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 5	<code>bool requiresNewOtra;</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 6	<code>bool suppressSinAmbiente;</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 7	<code>string aislamiento;</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 9	<code>using (var outer = new TransactionScope())</code> → Crea o participa en una transacción ambiental según la opción indicada.
Línea 10	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 11	<code>var outerTransaction = Transaction.Current</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 12	<code>?? throw new InvalidOperationException("No se creó la transacción ambiental.");</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 13	<code>outerId = outerTransaction.TransactionInformation.LocalIdentifier;</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 14	<code>aislamiento = outerTransaction.IsolationLevel.ToString();</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 16	<code>using (var required = new TransactionScope(TransactionScopeOption.Required))</code> → Crea o participa en una transacción ambiental según la opción indicada.

Línea 17	{ → Delimita el bloque sintáctico correspondiente.
Línea 18	requiredMisma = Transaction.Current?.TransactionInformation.LocalIdentifier == outerId; → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 19	required.Complete(); → Marca/confirma la unidad transaccional cuando todas las operaciones requeridas han terminado correctamente.
Línea 20	} → Delimita el bloque sintáctico correspondiente.
Línea 22	using (var requiresNew = new TransactionScope(TransactionScopeOption.RequiresNew)) → Crea o participa en una transacción ambiental según la opción indicada.
Línea 23	{ → Delimita el bloque sintáctico correspondiente.
Línea 24	requiresNewOtra = Transaction.Current?.TransactionInformation.LocalIdentifier != outerId; → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 25	requiresNew.Complete(); → Marca/confirma la unidad transaccional cuando todas las operaciones requeridas han terminado correctamente.
Línea 26	} → Delimita el bloque sintáctico correspondiente.
Línea 28	using (var suppressed = new TransactionScope(TransactionScopeOption.Suppress)) → Crea o participa en una transacción ambiental según la opción indicada.
Línea 29	{ → Delimita el bloque sintáctico correspondiente.
Línea 30	suppressSinAmbiente = Transaction.Current is null; → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 31	suppressed.Complete(); → Marca/confirma la unidad transaccional cuando todas las operaciones requeridas han terminado correctamente.
Línea 32	} → Delimita el bloque sintáctico correspondiente.
Línea 33	outer.Complete(); → Marca/confirma la unidad transaccional cuando todas las operaciones requeridas han terminado correctamente.
Línea 34	} → Delimita el bloque sintáctico correspondiente.
Línea 36	return new OpcionesTransactionScopeM5Dto(→ Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.
Línea 37	requiredMisma, requiresNewOtra, suppressSinAmbiente, → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 38	aislamiento, TransactionManager.DefaultTimeout.TotalSeconds); → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 39	} → Delimita el bloque sintáctico correspondiente.

Paso 5: Compilar y ejecutar

POWERSHELL

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet ef migrations has-pending-model-changes --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console --configuration Release
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release --no-build
```

La ejecución debe finalizar con 5.5 0K. Si una comprobación interna falla, la aplicación lanza una excepción y el punto no se considera correctamente reproducido.

Diagnóstico técnico

La práctica se considera correcta cuando se observan estas condiciones:

- `TransactionScopeAsyncFlowOption.Enabled` conserva `Transaction.Current` tras `await`.
- Los dos `DbContext` usan una misma `SqlConnection` abierta para no depender de promoción distribuida.
- `Required`, `RequiresNew` y `Suppress` se distinguen por su `Transaction.Current`.
- Omitir `Complete` revierte SQL.
- El efecto externo simulado permanece porque no participa en `System.Transactions`.

Reto resuelto y ampliación

Reto: Ejecuta el escenario con `Suppress` alrededor de una operación y verifica qué queda fuera de la transacción ambiental.

Para resolverlo, conserva la misma regla del módulo: primero define qué resultado funcional debe mantenerse; después captura SQL, número de comandos, tracking, historial de migraciones o estado transaccional según corresponda. No uses una medición temporal aislada como sustituto de la evidencia técnica.

Errores comunes

- Asumir que `ReadCommitted` permite lecturas sucias.
- Afirmar que `Snapshot` elimina todos los bloqueos y conflictos.
- Abrir varios recursos durables sin considerar posible promoción a transacción distribuida.
- Esperar que HTTP, colas o archivos se reviertan con `TransactionScope`.

Cierre acumulativo

1. Ejecuta `dotnet build` en la solución del punto.
2. Ejecuta la aplicación y comprueba el marcador `5.5 OK`.
3. Conserva las migraciones existentes; solo 5.2 añade el cambio de modelo oficial de M5.
4. Comprueba que `Application` continúa sin depender de Entity Framework Core ni de Infrastructure.
5. Antes de avanzar, relaciona el resultado con el SQL, la transacción, el historial o la prueba que lo demuestra.

Resultado esperado

Al finalizar el punto 5.5, `AceriaData` debe compilar, ejecutarse sobre SQL Server LocalDB y demostrar de forma observable el comportamiento descrito. El marcador final es:

5.5 OK

TEXT

Conexión con el siguiente punto

El siguiente estado es `5.6` y parte directamente de este proyecto completo.

Punto 5.6 — Migraciones en entornos de producción: estrategias y despliegue

Contexto del proyecto

Este punto continúa directamente `M05/PROYECTO/5.5`. La carpeta `M05/PROYECTO/5.6` contiene la solución completa `AceriaData.sln` y conserva las capas y migraciones existentes. El modelo oficial no cambia en este punto; se conserva la última migración real ya existente.

Objetivo práctico

Separar desarrollo y despliegue controlado mediante `IMigrator`, scripts revisables y migration bundles, conservando el historial real.

Paso 1: Abrir el estado completo del punto

```
cd M05/PROYECTO/5.6
dotnet restore AceriaData.sln
```

POWERSHELL

La restauración se ejecuta sobre la solución local del punto. No es necesario copiar archivos desde otros puntos: la carpeta ya contiene el estado acumulativo completo.

Paso 2: Identificar los archivos principales

En este punto se trabajan principalmente estos archivos:

- `src/AceriaData.Infrastructure/Repositories/MigracionesProduccionM5Repositorio.cs`
- `deployment/generate-production-artifacts.ps1`
- `deployment/PLAN_DESPLIEGUE.md`

Los archivos se estudian dentro de su capa real. Application conserva sus abstracciones y casos de uso; Infrastructure contiene EF Core y acceso a proveedor; Console compone y ejecuta; los tests se ubican fuera de las capas de producción cuando aparecen en 5.9 y 5.11.

Paso 3: Implementar y estudiar `MigracionesProduccionM5Repositorio.cs`

```
public async Task<MigracionesProduccionM5Dto> AplicarConIMigratorAsync()
{
    using var scope = _scopeFactory.CreateScope();
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();

    var migrator = context.GetService<IMigrator>();
    await migrator.MigrateAsync();

    var aplicadas = (await context.Database.GetAppliedMigrationsAsync()).ToArray();
    var pendientes = (await context.Database.GetPendingMigrationsAsync()).ToArray();
    var tablaExiste = await ExisteTablaHistorialAsync(context);

    return new MigracionesProduccionM5Dto(
        aplicadas.Length,
        pendientes.Length,
        aplicadas.LastOrDefault() ?? "<ninguna>",
        tablaExiste,
        "__EFMigrationsHistory");
}
```

CSHARP

Explicación línea a línea — `MigracionesProduccionM5Repositorio.cs`

Línea 1	<code>public async Task<MigracionesProduccionM5Dto> AplicarConIMigratorAsync()</code> → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 2	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 3	<code>using var scope = _scopeFactory.CreateScope();</code> → Crea un ámbito de DI independiente; los servicios Scoped, incluido <code>AceriaDbContext</code> , no se comparten con otros actores.
Línea 4	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve el <code>DbContext</code> del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.
Línea 6	<code>var migrator = context.GetService<IMigrator>();</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 7	<code>await migrator.MigrateAsync();</code> → Aplica la cadena real de migraciones al proveedor de pruebas o despliegue.
Línea 9	<code>var aplicadas = (await context.Database.GetAppliedMigrationsAsync()).ToArray();</code> → Lee la lista de migraciones que la base registra como aplicadas.

Línea 10	<code>var pendientes = (await context.Database.GetPendingMigrationsAsync()).ToArray();</code> → Comprueba si queda alguna migración del ensamblado pendiente de aplicar.
Línea 11	<code>var tablaExiste = await ExisteTablaHistorialAsync(context);</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 13	<code>return new MigracionesProduccionM5Dto()</code> → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.
Línea 14	<code>aplicadas.Length,</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 15	<code>pendientes.Length,</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 16	<code>aplicadas.LastOrDefault() ?? "<ninguna>"</code> , → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 17	<code>tablaExiste,</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 18	<code>"__EFMigrationsHistory");</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 19	<code>}</code> → Delimita el bloque sintáctico correspondiente.

Paso 4: Implementar y estudiar `generate-production-artifacts.ps1`

POWERSHELL

```

$ErrorActionPreference = "Stop"
$Artifacts = Join-Path $PSScriptRoot "artifacts"
New-Item -ItemType Directory -Force -Path $Artifacts | Out-Null

dotnet ef migrations script --idempotent `
  --project ../src/AceriaData.Infrastructure `
  --startup-project ../src/AceriaData.Console `
  --configuration Release `
  --output (Join-Path $Artifacts "aceria-idempotent.sql")

if ($LASTEXITCODE -ne 0) { throw "No se pudo generar el script idempotente" }

dotnet ef migrations bundle `
  --project ../src/AceriaData.Infrastructure `
  --startup-project ../src/AceriaData.Console `
  --configuration Release `
  --output (Join-Path $Artifacts "aceria-efbundle.exe") `
  --force

if ($LASTEXITCODE -ne 0) { throw "No se pudo generar el migration bundle" }

```

Explicación línea a línea — `generate-production-artifacts.ps1`

Línea 1	<code>\$ErrorActionPreference = "Stop"</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 2	<code>\$Artifacts = Join-Path \$PSScriptRoot "artifacts"</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 3	<code>New-Item -ItemType Directory -Force -Path \$Artifacts Out-Null</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 5	<code>`dotnet ef migrations script --idempotent ``</code> → Genera SQL desde la cadena real de migraciones de EF Core.
Línea 6	<code>`--project ../src/AceriaData.Infrastructure ``</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 7	<code>`--startup-project ../src/AceriaData.Console ``</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 8	<code>`--configuration Release ``</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 9	<code>--output (Join-Path \$Artifacts "aceria-idempotent.sql")</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 11	<code>if (\$LASTEXITCODE -ne 0) { throw "No se pudo generar el script idempotente" }</code> → Convierte un fallo del comando externo en un fallo explícito del script.
Línea 13	<code>`dotnet ef migrations bundle ``</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 14	<code>`--project ../src/AceriaData.Infrastructure ``</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 15	<code>`--startup-project ../src/AceriaData.Console ``</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 16	<code>`--configuration Release ``</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 17	<code>`--output (Join-Path \$Artifacts "aceria-efbundle.exe") ``</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 18	<code>--force</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 20	<code>if (\$LASTEXITCODE -ne 0) { throw "No se pudo generar el migration bundle" }</code> → Convierte un fallo del comando externo en un fallo explícito del script.

Paso 5: Implementar y estudiar Comandos de despliegue

```
dotnet ef migrations script --idempotent `
  --project src/AceriaData.Infrastructure `
  --startup-project src/AceriaData.Console `
  --configuration Release `
  --output aceria-idempotent.sql

# El bundle recibe la conexión en ejecución, no desde código fuente.
./aceria-efbundle.exe --connection $env:ACERIA_PROD_CONNECTION
```

POWERSHELL

Explicación línea a línea — Comandos de despliegue

Línea 1	<code>`dotnet ef migrations script --idempotent ``</code> → Genera SQL desde la cadena real de migraciones de EF Core.
Línea 2	<code>`--project src/AceriaData.Infrastructure ``</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 3	<code>`--startup-project src/AceriaData.Console ``</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 4	<code>`--configuration Release ``</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 5	<code>--output aceria-idempotent.sql</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 7	<code># El bundle recibe la conexión en ejecución, no desde código fuente.</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 8

./aceria-efbundle.exe --connection \$env:ACERIA_PROD_CONNECTION → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Paso 6: Compilar y ejecutar

POWERSHELL

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet ef migrations has-pending-model-changes --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console --configuration Release
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release --no-build
```

La ejecución debe finalizar con **5.6 OK**. Si una comprobación interna falla, la aplicación lanza una excepción y el punto no se considera correctamente reproducido.

Diagnóstico técnico

La práctica se considera correcta cuando se observan estas condiciones:

- MigrationsAssembly apunta al ensamblado real de migraciones.
- IMigrator aplica la cadena y deja cero migraciones pendientes.
- Se conserva __EFMigrationsHistory.
- El bundle recibe la cadena de conexión desde el entorno de despliegue.
- El plan distingue downgrade de recuperación de datos.

Reto resuelto y ampliación

Reto: Genera un script revisable y un bundle para la misma cadena y documenta qué control operacional aporta cada uno.

Para resolverlo, conserva la misma regla del módulo: primero define qué resultado funcional debe mantenerse; después captura SQL, número de comandos, tracking, historial de migraciones o estado transaccional según corresponda. No uses una medición temporal aislada como sustituto de la evidencia técnica.

Errores comunes

- Ejecutar Database.Migrate en cada réplica como regla general de producción.
- Renombrar MigrationsHistoryTable sin mover el historial existente.
- Guardar credenciales de producción en el repositorio.
- Confundir un Down destructivo con restauración segura.

Cierre acumulativo

1. Ejecuta `dotnet build` en la solución del punto.
2. Ejecuta la aplicación y comprueba el marcador **5.6 OK**.
3. Conserva las migraciones existentes; solo 5.2 añade el cambio de modelo oficial de M5.
4. Comprueba que Application continúa sin depender de Entity Framework Core ni de Infrastructure.
5. Antes de avanzar, relaciona el resultado con el SQL, la transacción, el historial o la prueba que lo demuestra.

Resultado esperado

Al finalizar el punto 5.6, AceriaData debe compilar, ejecutarse sobre SQL Server LocalDB y demostrar de forma observable el comportamiento descrito. El marcador final es:

5.6 OK

TEXT

Conexión con el siguiente punto

El siguiente estado es **5.7** y parte directamente de este proyecto completo.

Punto 5.7 — Migraciones idempotentes y scripts SQL

Contexto del proyecto

Este punto continúa directamente M05/PROYECTO/5.6. La carpeta M05/PROYECTO/5.7 contiene la solución completa `AceriaData.sln` y conserva las capas y migraciones existentes. El modelo oficial no cambia en este punto; se conserva la última migración real ya existente.

Objetivo práctico

Generar scripts completo, idempotente, de rango y downgrade, y demostrar idempotencia aplicando dos veces el mismo script.

Paso 1: Abrir el estado completo del punto

```
cd M05/PROYECTO/5.7
dotnet restore AceriaData.sln
```

POWERSHELL

La restauración se ejecuta sobre la solución local del punto. No es necesario copiar archivos desde otros puntos: la carpeta ya contiene el estado acumulativo completo.

Paso 2: Identificar los archivos principales

En este punto se trabajan principalmente estos archivos:

- `deployment/validate-idempotent-scripts.ps1`

Los archivos se estudian dentro de su capa real. Application conserva sus abstracciones y casos de uso; Infrastructure contiene EF Core y acceso a proveedor; Console compone y ejecuta; los tests se ubican fuera de las capas de producción cuando aparecen en 5.9 y 5.11.

Paso 3: Implementar y estudiar `validate-idempotent-scripts.ps1`

```
dotnet ef migrations script `
  --project $Infrastructure --startup-project $Startup `
  --configuration Release --output $Complete `

dotnet ef migrations script --idempotent `
  --project $Infrastructure --startup-project $Startup `
  --configuration Release --output $Idempotent `

dotnet ef migrations script M2_2_12_Architecture M5_5_2_ConcurrencyTokens `
  --project $Infrastructure --startup-project $Startup `
  --configuration Release --output $Range `

dotnet ef migrations script M5_5_2_ConcurrencyTokens M2_2_12_Architecture `
  --project $Infrastructure --startup-project $Startup `
  --configuration Release --output $Downgrade `

$idempotentSql = Get-Content $Idempotent -Raw
if ($idempotentSql -notmatch "_EFMigrationsHistory") { throw "Falta historial" }
if ($idempotentSql -notmatch "IF NOT EXISTS") { throw "Faltan guardas" }
if ($idempotentSql -notmatch "M5_5_2_ConcurrencyTokens") { throw "Falta migración final" }
```

POWERSHELL

Explicación línea a línea — `validate-idempotent-scripts.ps1`

Línea 1	<code>`dotnet ef migrations script ``</code> → Genera SQL desde la cadena real de migraciones de EF Core.
Línea 2	<code>`--project \$Infrastructure --startup-project \$Startup ``</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 3	<code>--configuration Release --output \$Complete</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 5	<code>`dotnet ef migrations script --idempotent ``</code> → Genera SQL desde la cadena real de migraciones de EF Core.

Línea 6	<code>`--project \$Infrastructure --startup-project \$Startup ``</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 7	<code>--configuration Release --output \$Idempotent</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 9	<code>`dotnet ef migrations script M2_2_12_Architecture M5_5_2_ConcurrencyTokens ``</code> → Genera SQL desde la cadena real de migraciones de EF Core.
Línea 10	<code>`--project \$Infrastructure --startup-project \$Startup ``</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 11	<code>--configuration Release --output \$Range</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 13	<code>`dotnet ef migrations script M5_5_2_ConcurrencyTokens M2_2_12_Architecture ``</code> → Genera SQL desde la cadena real de migraciones de EF Core.
Línea 14	<code>`--project \$Infrastructure --startup-project \$Startup ``</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 15	<code>--configuration Release --output \$Downgrade</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 17	<code>\$idempotentSql = Get-Content \$Idempotent -Raw</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 18	<code>if (\$idempotentSql -notmatch "__EFMigrationsHistory") { throw "Falta historial" }</code> → Evalúa una condición necesaria y detiene o desvía el flujo cuando no se cumple.
Línea 19	<code>if (\$idempotentSql -notmatch "IF NOT EXISTS") { throw "Faltan guardas" }</code> → Evalúa una condición necesaria y detiene o desvía el flujo cuando no se cumple.
Línea 20	<code>if (\$idempotentSql -notmatch "M5_5_2_ConcurrencyTokens") { throw "Falta migración final" }</code> → Evalúa una condición necesaria y detiene o desvía el flujo cuando no se cumple.

Paso 4: Implementar y estudiar `validate-idempotent-scripts.ps1`

POWERSHELL

```

Write-Host "== Primera aplicacion del script idempotente =="
& sqlcmd -S $Server -d $Database -E -I -b -i $Idempotent
Assert-LastExitCode "La primera aplicacion fallo"

$count1 = [int](Read-Scalar "SELECT COUNT(*) FROM dbo.__EFMigrationsHistory;")
if ($count1 -le 0) { throw "El historial quedó vacío" }

$lastMigration = Read-Scalar "SELECT TOP (1) MigrationId FROM dbo.__EFMigrationsHistory ORDER BY MigrationId DESC;"
if ($lastMigration -notmatch "M5_5_2_ConcurrencyTokens") {
    throw "La migración final no es la esperada: $lastMigration"
}

Write-Host "== Segunda aplicacion del mismo script idempotente =="
& sqlcmd -S $Server -d $Database -E -I -b -i $Idempotent
Assert-LastExitCode "La segunda aplicacion fallo"

$count2 = [int](Read-Scalar "SELECT COUNT(*) FROM dbo.__EFMigrationsHistory;")
if ($count2 -ne $count1) {
    throw "La segunda aplicación alteró el número de migraciones"
}

Write-Host "5.7 IDEMPOTENCIA OK"

```

Explicación línea a línea — `validate-idempotent-scripts.ps1`

Línea 1	<code>Write-Host "== Primera aplicacion del script idempotente =="</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
----------------	--

Línea 2	<code>& sqlcmd -S \$Server -d \$Database -E -I -b -i \$Idempotent</code> → Ejecuta o consulta SQL Server desde PowerShell para validar el artefacto generado.
Línea 3	<code>Assert-LastExitCode "La primera aplicacion fallo"</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 5	<code>\$count1 = [int](Read-Scalar "SELECT COUNT(*) FROM dbo.__EFMigrationsHistory;")</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 6	<code>if (\$count1 -le 0) { throw "El historial quedó vacío" }</code> → Evalúa una condición necesaria y detiene o desvía el flujo cuando no se cumple.
Línea 8	<code>\$lastMigration = Read-Scalar "SELECT TOP (1) MigrationId FROM dbo.__EFMigrationsHistory ORDER BY MigrationId DESC;"</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 9	<code>if (\$lastMigration -notmatch "M5_5_2_ConcurrencyTokens") {</code> → Evalúa una condición necesaria y detiene o desvía el flujo cuando no se cumple.
Línea 10	<code>throw "La migración final no es la esperada: \$lastMigration"</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 11	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 13	<code>Write-Host "== Segunda aplicacion del mismo script idempotente =="</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 14	<code>& sqlcmd -S \$Server -d \$Database -E -I -b -i \$Idempotent</code> → Ejecuta o consulta SQL Server desde PowerShell para validar el artefacto generado.
Línea 15	<code>Assert-LastExitCode "La segunda aplicacion fallo"</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 17	<code>\$count2 = [int](Read-Scalar "SELECT COUNT(*) FROM dbo.__EFMigrationsHistory;")</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 18	<code>if (\$count2 -ne \$count1) {</code> → Evalúa una condición necesaria y detiene o desvía el flujo cuando no se cumple.
Línea 19	<code>throw "La segunda aplicación alteró el número de migraciones"</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 20	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 22	<code>Write-Host "5.7 IDEMPOTENCIA OK"</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Paso 5: Compilar y ejecutar

POWERSHELL

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet ef migrations has-pending-model-changes --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console --configuration Release
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release --no-build
./deployment/validate-idempotent-scripts.ps1
```

La ejecución debe finalizar con **5.7 OK**. Si una comprobación interna falla, la aplicación lanza una excepción y el punto no se considera correctamente reproducido.

Diagnóstico técnico

La práctica se considera correcta cuando se observan estas condiciones:

- El script idempotente consulta `__EFMigrationsHistory`.
- La primera aplicación deja la migración final esperada.
- La segunda aplicación no cambia el número de filas del historial.
- El esquema esperado permanece estable.
- El downgrade se trata como operación potencialmente destructiva.

Reto resuelto y ampliación

Reto: Crea una base aislada adicional y aplica primero un script de rango y después el idempotente completo; compara el historial final.

Para resolverlo, conserva la misma regla del módulo: primero define qué resultado funcional debe mantenerse; después captura SQL, número de comandos, tracking, historial de migraciones o estado transaccional según corresponda. No uses una medición temporal aislada como sustituto de la evidencia técnica.

Errores comunes

- Llamar idempotente a un script sin ejecutarlo dos veces.
- Usar nombres ficticios de migración que no existen en el proyecto.
- Aplicar un downgrade sin revisar operaciones destructivas.
- Tomar el script generado como sustituto del código de migraciones.

Cierre acumulativo

1. Ejecuta `dotnet build` en la solución del punto.
2. Ejecuta la aplicación y comprueba el marcador `5.7 OK`.
3. Conserva las migraciones existentes; solo 5.2 añade el cambio de modelo oficial de M5.
4. Comprueba que Application continúa sin depender de Entity Framework Core ni de Infrastructure.
5. Antes de avanzar, relaciona el resultado con el SQL, la transacción, el historial o la prueba que lo demuestra.

Resultado esperado

Al finalizar el punto 5.7, AceriaData debe compilar, ejecutarse sobre SQL Server LocalDB y demostrar de forma observable el comportamiento descrito. El marcador final es:

5.7 OK

TEXT

Conexión con el siguiente punto

El siguiente estado es 5.8 y parte directamente de este proyecto completo.

Punto 5.8 — Migraciones en equipos: conflictos y buenas prácticas

Contexto del proyecto

Este punto continúa directamente `M05/PROYECTO/5.7`. La carpeta `M05/PROYECTO/5.8` contiene la solución completa `AceriaData.sln` y conserva las capas y migraciones existentes. El modelo oficial no cambia en este punto; se conserva la última migración real ya existente.

Objetivo práctico

Reproducir migraciones paralelas y resolverlas regenerando la migración propia sobre el modelo ya fusionado.

Paso 1: Abrir el estado completo del punto

```
cd M05/PROYECTO/5.8
dotnet restore AceriaData.sln
```

POWERSHELL

La restauración se ejecuta sobre la solución local del punto. No es necesario copiar archivos desde otros puntos: la carpeta ya contiene el estado acumulativo completo.

Paso 2: Identificar los archivos principales

En este punto se trabajan principalmente estos archivos:

- `team-migrations/validate-team-migrations.ps1`

Los archivos se estudian dentro de su capa real. Application conserva sus abstracciones y casos de uso; Infrastructure contiene EF Core y acceso a proveedor; Console compone y ejecuta; los tests se ubican fuera de las capas de producción cuando aparecen en 5.9 y 5.11.

Paso 3: Implementar y estudiar `validate-team-migrations.ps1`

El script crea primero dos copias temporales y desechables del mismo estado inicial. A continuación aplica en cada copia un cambio de modelo distinto y genera las migraciones paralelas:

```
Write-Host "== Rama A desde estado inicial =="
Add-TeamProperty $BranchA "EquipoRevisionA"
Add-Migration $BranchA "M5_5_8_TeamA"

Write-Host "== Rama B paralela desde el mismo estado inicial =="
Add-TeamProperty $BranchB "EquipoRevisionB"
Add-Migration $BranchB "M5_5_8_TeamBParallel"

$designerB = Get-ChildItem `
    (Join-Path $BranchB "src/AceriaData.Infrastructure/Migrations") `
    -Filter "*_M5_5_8_TeamBParallel.Designer.cs" | Select-Object -First 1
$parallelMetadata = Get-Content $designerB.FullName -Raw
if ($parallelMetadata -match "EquipoRevisionA") {
    throw "La rama B paralela no debería conocer el cambio A"
}
if ($parallelMetadata -notmatch "EquipoRevisionB") {
    throw "La rama B paralela no contiene su propio cambio"
}
Write-Host "Renombrar B no fusionaría sus metadatos."
```

POWERSHELL

Explicación línea a línea — `validate-team-migrations.ps1`

Línea 1	<code>Write-Host "== Rama A desde estado inicial =="</code> → Identifica en la salida la preparación de la primera rama temporal.
Línea 2	<code>Add-TeamProperty \$BranchA "EquipoRevisionA"</code> → Añade el cambio de modelo propio de la rama A.
Línea 3	<code>Add-Migration \$BranchA "M5_5_8_TeamA"</code> → Genera la migración A desde el estado inicial de esa rama.
Línea 5	<code>Write-Host "== Rama B paralela desde el mismo estado inicial =="</code> → Identifica la segunda rama, creada desde el mismo estado que A.
Línea 6	<code>Add-TeamProperty \$BranchB "EquipoRevisionB"</code> → Añade a B un cambio diferente sin incorporar todavía A.
Línea 7	<code>Add-Migration \$BranchB "M5_5_8_TeamBParallel"</code> → Genera la migración paralela B para poder inspeccionar su metadata.
Línea 11	<code>`\$designerB = Get-ChildItem ``</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 12	<code>`(Join-Path \$BranchB "src/AceriaData.Infrastructure/Migrations") ``</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 13	<code>-Filter "*_M5_5_8_TeamBParallel.Designer.cs" Select-Object -First 1</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 14	<code>\$parallelMetadata = Get-Content \$designerB.FullName -Raw</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 15	<code>if (\$parallelMetadata -match "EquipoRevisionA") {</code> → Evalúa una condición necesaria y detiene o desvía el flujo cuando no se cumple.
Línea 16	<code>throw "La rama B paralela no debería conocer el cambio A"</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 17	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 18	<code>if (\$parallelMetadata -notmatch "EquipoRevisionB") {</code> → Evalúa una condición necesaria y detiene o desvía el flujo cuando no se cumple.
Línea 19	<code>throw "La rama B paralela no contiene su propio cambio"</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 20	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 21	<code>Write-Host "Renombrar B no fusionaría sus metadatos."</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Paso 4: Implementar y estudiar `validate-team-migrations.ps1`

POWERSHELL

```
Write-Host "== Estado fusionado correcto: incorporar A y regenerar B =="
Copy-Item $BranchA -Destination $Merged -Recurse -Force
Add-TeamProperty $Merged "EquipoRevisionB"
Add-Migration $Merged "M5_5_8_TeamBRegenerated"

$designerRegenerated = Get-ChildItem `
    (Join-Path $Merged "src/AceriaData.Infrastructure/Migrations") `
    -Filter "*_M5_5_8_TeamBRegenerated.Designer.cs" | Select-Object -First 1
$mergedMetadata = Get-Content $designerRegenerated.FullName -Raw
if ($mergedMetadata -notmatch "EquipoRevisionA" -or
    $mergedMetadata -notmatch "EquipoRevisionB") {
    throw "La migración regenerada no representa el modelo fusionado A+B"
}

dotnet ef migrations has-pending-model-changes `
    --project src/AceriaData.Infrastructure `
    --startup-project src/AceriaData.Console `
    --configuration Release

Write-Host "5.8 EQUIPOS OK"
```

Explicación línea a línea — `validate-team-migrations.ps1`

Línea 1	<code>Write-Host "== Estado fusionado correcto: incorporar A y regenerar B =="</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 2	<code>Copy-Item \$BranchA -Destination \$Merged -Recurse -Force</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 3	<code>Add-TeamProperty \$Merged "EquipoRevisionB"</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 4	<code>Add-Migration \$Merged "M5_5_8_TeamBRegenerated"</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 6	<code>`\$designerRegenerated = Get-ChildItem ``</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 7	<code>`(Join-Path \$Merged "src/AceriaData.Infrastructure/Migrations") `</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 8	<code>-Filter "*" _M5_5_8_TeamBRegenerated.Designer.cs" Select-Object -First 1</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 9	<code>\$mergedMetadata = Get-Content \$designerRegenerated.FullName -Raw</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 10	<code>if (\$mergedMetadata -notmatch "EquipoRevisionA" -or</code> → Evalúa una condición necesaria y detiene o desvía el flujo cuando no se cumple.
Línea 11	<code>\$mergedMetadata -notmatch "EquipoRevisionB") {</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 12	<code>throw "La migración regenerada no representa el modelo fusionado A+B"</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 13	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 15	<code>`dotnet ef migrations has-pending-model-changes `</code> → Comprueba que el modelo actual coincide con el snapshot de migraciones.
Línea 16	<code>`--project src/AceriaData.Infrastructure `</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 17	<code>`--startup-project src/AceriaData.Console `</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 18	<code>--configuration Release</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 20	<code>Write-Host "5.8 EQUIPOS OK"</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Paso 5: Compilar y ejecutar

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet ef migrations has-pending-model-changes --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console --configuration Release
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release --no-build ./team-migrations/validate-team-migrations.ps1
```

POWERSHELL

La ejecución debe finalizar con `5.8 OK`. Si una comprobación interna falla, la aplicación lanza una excepción y el punto no se considera correctamente reproducido.

Diagnóstico técnico

La práctica se considera correcta cuando se observan estas condiciones:

- Rama A y B parten del mismo estado y generan migraciones independientes.
- El Designer de B paralela desconoce el cambio A.
- Renombrar B no corrige sus metadatos.
- La B regenerada sobre A contiene ambos cambios.
- has-pending-model-changes y SQL Server verifican el resultado fusionado.

Reto resuelto y ampliación

Reto: Simula una tercera rama C y describe el orden seguro para integrar A, B y C sin reescribir migraciones ya publicadas.

Para resolverlo, conserva la misma regla del módulo: primero define qué resultado funcional debe mantenerse; después captura SQL, número de comandos, tracking, historial de migraciones o estado transaccional según corresponda. No uses una medición temporal aislada como sustituto de la evidencia técnica.

Errores comunes

- Resolver el conflicto renombrando archivos o timestamps.
- Borrar una migración que ya fue compartida o aplicada sin coordinación.
- Editar manualmente Designer o snapshot para aparentar coherencia.
- No ejecutar `has-pending-model-changes` tras el merge.

Cierre acumulativo

1. Ejecuta `dotnet build` en la solución del punto.
2. Ejecuta la aplicación y comprueba el marcador `5.8 OK`.
3. Conserva las migraciones existentes; solo 5.2 añade el cambio de modelo oficial de M5.
4. Comprueba que Application continúa sin depender de Entity Framework Core ni de Infrastructure.
5. Antes de avanzar, relaciona el resultado con el SQL, la transacción, el historial o la prueba que lo demuestra.

Resultado esperado

Al finalizar el punto 5.8, AceriaData debe compilar, ejecutarse sobre SQL Server LocalDB y demostrar de forma observable el comportamiento descrito. El marcador final es:

5.8 OK

TEXT

Conexión con el siguiente punto

El siguiente estado es 5.9 y parte directamente de este proyecto completo.

Punto 5.9 — Patrón Repositorio y Unidad de Trabajo en aplicaciones empresariales

Contexto del proyecto

Este punto continúa directamente `M05/PROYECTO/5.8`. La carpeta `M05/PROYECTO/5.9` contiene la solución completa `AceriaData.sln` y conserva las capas y migraciones existentes. El modelo oficial no cambia en este punto; se conserva la última migración real ya existente.

Objetivo práctico

Consolidar Repository/UoW como decisión arquitectónica de AceriaData, probarla con Moq y comparar funcionalmente con acceso directo.

Paso 1: Abrir el estado completo del punto

```
cd M05/PROYECTO/5.9
dotnet restore AceriaData.sln
```

POWERSHELL

La restauración se ejecuta sobre la solución local del punto. No es necesario copiar archivos desde otros puntos: la carpeta ya contiene el estado acumulativo completo.

Paso 2: Identificar los archivos principales

En este punto se trabajan principalmente estos archivos:

- `src/AceriaData.Infrastructure/Repositories/RepositoryPattern.cs`

- tests/AceriaData.Tests/RepositoryPatternTests.cs

Los archivos se estudian dentro de su capa real. Application conserva sus abstracciones y casos de uso; Infrastructure contiene EF Core y acceso a proveedor; Console compone y ejecuta; los tests se ubican fuera de las capas de producción cuando aparecen en 5.9 y 5.11.

Paso 3: Implementar y estudiar RepositoryPattern.cs

```
public class Repositorio<T> : IRepository<T> where T : class
{
    protected readonly AceriaDbContext _context;
    protected readonly DbSet<T> _dbSet;

    public Repositorio(AceriaDbContext context)
    {
        _context = context;
        _dbSet = context.Set<T>();
    }

    public virtual T? ObtenerPorId(int id) => _dbSet.Find(id);
    public virtual List<T> ObtenerTodas() => _dbSet.ToList();
    public virtual void Agregar(T entidad) => _dbSet.Add(entidad);
    public virtual void Eliminar(T entidad) => _dbSet.Remove(entidad);
}

public sealed class DetalleOrdenRepositorio : Repositorio<DetalleOrden>, IDetalleOrdenRepositorio
{
    public DetalleOrdenRepositorio(AceriaDbContext context) : base(context) { }

    public DetalleOrden? ObtenerPorOrden(int ordenId) =>
        _context.DetallesOrden.AsNoTracking().FirstOrDefault(d => d.OrdenId == ordenId);

    public List<DetalleOrden> ObtenerConTemperaturaMayorA(double temperatura) =>
        _context.DetallesOrden.AsNoTracking()
            .Where(d => d.TemperaturaColada > temperatura)
            .OrderBy(d => d.Id)
            .ToList();
}
```

CSHARP

Explicación línea a línea — RepositoryPattern.cs

Línea 1	public class Repositorio<T> : IRepository<T> where T : class → Declara la clase que encapsula la responsabilidad mostrada en este ejemplo.
Línea 2	{ → Delimita el bloque sintáctico correspondiente.
Línea 3	protected readonly AceriaDbContext _context; → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 4	protected readonly DbSet<T> _dbSet; → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 6	public Repositorio(AceriaDbContext context) → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 7	{ → Delimita el bloque sintáctico correspondiente.
Línea 8	_context = context; → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 9	_dbSet = context.Set<T>(); → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 10	} → Delimita el bloque sintáctico correspondiente.
Línea 12	public virtual T? ObtenerPorId(int id) => _dbSet.Find(id); → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 13	public virtual List<T> ObtenerTodas() => _dbSet.ToList(); → Materializa o agrega la consulta en este punto, provocando la ejecución contra el proveedor cuando procede.

Línea 14	<code>public virtual void Agregar(T entidad) => _dbSet.Add(entidad);</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 15	<code>public virtual void Eliminar(T entidad) => _dbSet.Remove(entidad);</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 16	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 18	<code>public sealed class DetalleOrdenRepositorio : Repositorio<DetalleOrden>, IDetalleOrdenRepositorio</code> → Declara la clase que encapsula la responsabilidad mostrada en este ejemplo.
Línea 19	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 20	<code>public DetalleOrdenRepositorio(AceriaDbContext context) : base(context) { }</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 22	<code>public DetalleOrden? ObtenerPorOrden(int ordenId) =></code> → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 23	<code>_context.DetallesOrden.AsNoTracking().FirstOrDefault(d => d.OrdenId == ordenId);</code> → Ejecuta una lectura sin dejar entidades en el ChangeTracker.
Línea 25	<code>public List<DetalleOrden> ObtenerConTemperaturaMayorA(double temperatura) =></code> → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 26	<code>_context.DetallesOrden.AsNoTracking()</code> → Ejecuta una lectura sin dejar entidades en el ChangeTracker.
Línea 27	<code>.Where(d => d.TemperaturaColada > temperatura)</code> → Añade el predicado a la consulta; EF Core intentará traducirlo al proveedor antes de materializar.
Línea 28	<code>.OrderBy(d => d.Id)</code> → Añade orden determinista a la consulta antes de materializar.
Línea 29	<code>.ToList();</code> → Materializa o agrega la consulta en este punto, provocando la ejecución contra el proveedor cuando procede.
Línea 30	<code>}</code> → Delimita el bloque sintáctico correspondiente.

Paso 4: Implementar y estudiar `RepositoryPatternTests.cs`

CSHARP

```
[Fact]
public void ObtenerPendientes_UsaLaAbstraccionSinBaseDeDatos()
{
    var repo = new Mock<IOrdenRepositorio>();
    repo.Setup(r => r.ObtenerTodas()).Returns(
        [
            new OrdenFabricacion { NumeroOrden = "OF-1", Estado = "Pendiente" },
            new OrdenFabricacion { NumeroOrden = "OF-2", Estado = "EnProceso" },
            new OrdenFabricacion { NumeroOrden = "OF-3", Estado = "Pendiente" }
        ]
    );

    var servicio = new OrdenesConsultaM5Service(repo.Object);
    var pendientes = servicio.ObtenerPendientes();

    Assert.Equal(2, pendientes.Count);
    repo.Verify(r => r.ObtenerTodas(), Times.Once);
}

[Fact]
public void Registrar_DelegaEnElRepositorio()
{
    var repo = new Mock<IOrdenRepositorio>();
    var servicio = new OrdenesConsultaM5Service(repo.Object);
    var orden = new OrdenFabricacion { NumeroOrden = "OF-MOCK", Estado = "Pendiente" };

    servicio.Registrar(orden);
}
```

```
repo.Verify(r => r.Agregar(orden), Times.Once);
}
```

Explicación línea a línea — RepositoryPatternTests.cs

Línea 1	[Fact] → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 2	public void ObtenerPendientes_UsaLaAbstraccionSinBaseDeDatos() → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 3	{ → Delimita el bloque sintáctico correspondiente.
Línea 4	var repo = new Mock<IOrdenRepositorio>(); → Crea un mock de una abstracción de Application, sin necesitar EF Core ni una base de datos.
Línea 5	repo.Setup(r => r.ObtenerTodas()).Returns(→ Define el comportamiento que devolverá el mock durante el caso de prueba.
Línea 6	[→ Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 7	new OrdenFabricacion { NumeroOrden = "OF-1", Estado = "Pendiente" }, → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 8	new OrdenFabricacion { NumeroOrden = "OF-2", Estado = "EnProceso" }, → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 9	new OrdenFabricacion { NumeroOrden = "OF-3", Estado = "Pendiente" } → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 10	]); → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 12	var servicio = new OrdenesConsultaM5Service(repo.Object); → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 13	var pendientes = servicio.ObtenerPendientes(); → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 15	Assert.Equal(2, pendientes.Count); → Expresa una condición observable que la prueba exige para considerar correcto el comportamiento.
Línea 16	repo.Verify(r => r.ObtenerTodas(), Times.Once); → Comprueba que la colaboración con la dependencia ocurrió exactamente como espera el caso de uso.
Línea 17	} → Delimita el bloque sintáctico correspondiente.
Línea 19	[Fact] → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 20	public void Registrar_DelegaEnElRepositorio() → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 21	{ → Delimita el bloque sintáctico correspondiente.
Línea 22	var repo = new Mock<IOrdenRepositorio>(); → Crea un mock de una abstracción de Application, sin necesitar EF Core ni una base de datos.
Línea 23	var servicio = new OrdenesConsultaM5Service(repo.Object); → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 24	var orden = new OrdenFabricacion { NumeroOrden = "OF-MOCK", Estado = "Pendiente" }; → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 26	servicio.Registrar(orden); → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 28	<code>repo.Verify(r => r.Agregar(orden), Times.Once);</code> → Comprueba que la colaboración con la dependencia ocurrió exactamente como espera el caso de uso.
Línea 29	<code>}</code> → Delimita el bloque sintáctico correspondiente.

Paso 5: Compilar y ejecutar

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet ef migrations has-pending-model-changes --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console --configuration Release
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release --no-build
dotnet test tests/AceriaData.Tests/AceriaData.Tests.csproj --configuration Release --no-build
```

POWERSHELL

La ejecución debe finalizar con `5.9 OK`. Si una comprobación interna falla, la aplicación lanza una excepción y el punto no se considera correctamente reproducido.

Diagnóstico técnico

La práctica se considera correcta cuando se observan estas condiciones:

- Application trabaja con interfaces propias y no depende de EF Core.
- IUnidadDeTrabajo coordina repositorios con un SaveChanges.
- IDbContextFactory crea contextos independientes bajo demanda.
- Moq prueba lógica de Application sin base de datos.
- No se atribuye un coste fijo de tiempo o memoria al patrón.

Reto resuelto y ampliación

Reto: Añade una operación específica de negocio al repositorio de órdenes y prueba con Moq que el caso de uso la invoca una sola vez.

Para resolverlo, conserva la misma regla del módulo: primero define qué resultado funcional debe mantenerse; después captura SQL, número de comandos, tracking, historial de migraciones o estado transaccional según corresponda. No uses una medición temporal aislada como sustituto de la evidencia técnica.

Errores comunes

- Convertir Repository en obligación universal con EF Core.
- Exponer una abstracción que solo replique toda la API de DbSet sin aportar frontera.
- Mockear DbSet/LINQ cuando puede mockearse una interfaz de aplicación.
- Publicar un benchmark sin calentamiento, aislamiento y varias iteraciones.

Cierre acumulativo

1. Ejecuta `dotnet build` en la solución del punto.
2. Ejecuta la aplicación y comprueba el marcador `5.9 OK`.
3. Conserva las migraciones existentes; solo 5.2 añade el cambio de modelo oficial de M5.
4. Comprueba que Application continúa sin depender de Entity Framework Core ni de Infrastructure.
5. Antes de avanzar, relaciona el resultado con el SQL, la transacción, el historial o la prueba que lo demuestra.

Resultado esperado

Al finalizar el punto 5.9, AceriaData debe compilar, ejecutarse sobre SQL Server LocalDB y demostrar de forma observable el comportamiento descrito. El marcador final es:

5.9 OK

TEXT

Conexión con el siguiente punto

El siguiente estado es 5.10 y parte directamente de este proyecto completo.

Punto 5.10 — Logging y diagnóstico en Entity Framework Core

Contexto del proyecto

Este punto continúa directamente M05/PROYECTO/5.9. La carpeta M05/PROYECTO/5.10 contiene la solución completa AceriaData.sln y conserva las capas y migraciones existentes. El modelo oficial no cambia en este punto; se conserva la última migración real ya existente.

Objetivo práctico

Configurar logging estructurado, rotación, DiagnosticListener, EventCounters y telemetría para una aplicación Console/Worker.

Paso 1: Abrir el estado completo del punto

```
cd M05/PROYECTO/5.10
dotnet restore AceriaData.sln
```

POWERSHELL

La restauración se ejecuta sobre la solución local del punto. No es necesario copiar archivos desde otros puntos: la carpeta ya contiene el estado acumulativo completo.

Paso 2: Identificar los archivos principales

En este punto se trabajan principalmente estos archivos:

- src/AceriaData.Console/Diagnostics/EfDiagnosticObserver.cs
- src/AceriaData.Console/Diagnostics/EfEventCounterListener.cs
- src/AceriaData.Console/Logging/DiagnosticoM5Runner.cs

Los archivos se estudian dentro de su capa real. Application conserva sus abstracciones y casos de uso; Infrastructure contiene EF Core y acceso a proveedor; Console compone y ejecuta; los tests se ubican fuera de las capas de producción cuando aparecen en 5.9 y 5.11.

Paso 3: Implementar y estudiar EfDiagnosticObserver.cs

```
public sealed class EfDiagnosticObserver :
    IObservable<DiagnosticListener>,
    IObservable<KeyValuePair<string, object?>>,
    IDisposable
{
    private readonly ConcurrentBag<IDisposable> _subscriptions = new();
    private readonly IDisposable _allListenersSubscription;
    private int _efEventCount;

    public EfDiagnosticObserver()
    {
        _allListenersSubscription = DiagnosticListener.AllListeners.Subscribe(this);
    }

    public int EfEventCount => Volatile.Read(ref _efEventCount);

    public void OnNext(DiagnosticListener listener)
    {
        if (listener.Name == "Microsoft.EntityFrameworkCore")
            _subscriptions.Add(listener.Subscribe(this, IsEnabled));
    }

    private static bool IsEnabled(string eventName, object? arg1, object? arg2) =>
        eventName.Contains("Command", StringComparison.Ordinal) ||
        eventName.Contains("SaveChanges", StringComparison.Ordinal);

    public void OnNext(KeyValuePair<string, object?> value)
    {
        if (value.Key.StartsWith("Microsoft.EntityFrameworkCore.", StringComparison.Ordinal))
            Interlocked.Increment(ref _efEventCount);
    }
}
```

CSHARP

```

    }

    public void OnError(Exception error) { }
    public void OnCompleted() { }

    public void Dispose()
    {
        foreach (var subscription in _subscriptions)
            subscription.Dispose();
        _allListenersSubscription.Dispose();
    }
}

```

Explicación línea a línea — EfDiagnosticObserver.cs

Línea 1	<code>public sealed class EfDiagnosticObserver :</code> → Declara la clase que encapsula la responsabilidad mostrada en este ejemplo.
Línea 2	<code>IObserver<DiagnosticListener>,</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 3	<code>IObserver<KeyValuePair<string, object?>>,</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 4	<code>IDisposable</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 5	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 6	<code>private readonly ConcurrentBag<IDisposable> _subscriptions = new();</code> → Declara una operación auxiliar usada para mantener aislada la lógica del ejemplo.
Línea 7	<code>private readonly IDisposable _allListenersSubscription;</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 8	<code>private int _efEventCount;</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 10	<code>public EfDiagnosticObserver()</code> → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 11	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 12	<code>_allListenersSubscription = DiagnosticListener.AllListeners.Subscribe(this);</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 13	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 15	<code>public int EfEventCount => Volatile.Read(ref _efEventCount);</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 17	<code>public void OnNext(DiagnosticListener listener)</code> → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 18	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 19	<code>if (listener.Name == "Microsoft.EntityFrameworkCore")</code> → Evalúa una condición necesaria y detiene o desvía el flujo cuando no se cumple.
Línea 20	<code>_subscriptions.Add(listener.Subscribe(this, IsEnabled));</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 21	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 23	<code>private static bool IsEnabled(string eventName, object? arg1, object? arg2) =></code> → Declara una operación auxiliar usada para mantener aislada la lógica del ejemplo.

Línea 24	<code>eventName.Contains("Command", StringComparison.Ordinal) </code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 25	<code>eventName.Contains("SaveChanges", StringComparison.Ordinal);</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 27	<code>public void OnNext(KeyValuePair<string, object?> value)</code> → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 28	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 29	<code>if (value.Key.StartsWith("Microsoft.EntityFrameworkCore.", StringComparison.Ordinal))</code> → Evalúa una condición necesaria y detiene o desvía el flujo cuando no se cumple.
Línea 30	<code>Interlocked.Increment(ref _efEventCount);</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 31	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 33	<code>public void OnError(Exception error) { }</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 34	<code>public void OnCompleted() { }</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 36	<code>public void Dispose()</code> → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 37	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 38	<code>foreach (var subscription in _subscriptions)</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 39	<code>subscription.Dispose();</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 40	<code>_allListenersSubscription.Dispose();</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 41	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 42	<code>}</code> → Delimita el bloque sintáctico correspondiente.

Paso 4: Implementar y estudiar `EfEventCounterListener.cs`

```

public sealed class EfEventCounterListener : EventListener
{
    private readonly ConcurrentDictionary<string, double> _values = new(StringComparer.Ordinal);

    protected override void OnEventSourceCreated(EventSource eventSource)
    {
        if (eventSource.Name == "Microsoft.EntityFrameworkCore")
        {
            EnableEvents(
                eventSource,
                EventLevel.LogAlways,
                EventKeywords.All,
                new Dictionary<string, string?> { ["EventCounterIntervalSec"] = "1" });
        }
    }

    protected override void OnEventWritten(EventWrittenEventArgs eventData)
    {
        if (eventData.EventName != "EventCounters" || eventData.Payload is null || eventData.Payload.Count == 0)
            return;
        if (eventData.Payload[0] is not IDictionary<string, object> payload)
            return;
    }
}

```

CSHARP

```

if (!payload.TryGetValue("Name", out var nameValue) || nameValue is not string name)
    return;

double? value = null;
if (payload.TryGetValue("Mean", out var mean) && mean is double meanValue)
    value = meanValue;
else if (payload.TryGetValue("Increment", out var increment) && increment is double incrementValue)
    value = incrementValue;

if (value.HasValue)
    _values[name] = value.Value;
}
}

```

Explicación línea a línea — EfEventCounterListener.cs

Línea 1	<code>public sealed class EfEventCounterListener : EventListener</code> → Declara la clase que encapsula la responsabilidad mostrada en este ejemplo.
Línea 2	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 3	<code>private readonly ConcurrentDictionary<string, double> _values = new(StringComparer.Ordinal);</code> → Declara una operación auxiliar usada para mantener aislada la lógica del ejemplo.
Línea 5	<code>protected override void OnEventSourceCreated(EventSource eventSource)</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 6	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 7	<code>if (eventSource.Name == "Microsoft.EntityFrameworkCore")</code> → Evalúa una condición necesaria y detiene o desvía el flujo cuando no se cumple.
Línea 8	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 9	<code>EnableEvents(</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 10	<code>eventSource,</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 11	<code>EventLevel.LogAlways,</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 12	<code>EventKeywords.All,</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 13	<code>new Dictionary<string, string?> { ["EventCounterIntervalSec"] = "1" });</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 14	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 15	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 17	<code>protected override void OnEventWritten(EventWrittenEventArgs eventData)</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 18	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 19	<code>if (eventData.EventName != "EventCounters" eventData.Payload is null eventData.Payload.Count == 0)</code> → Evalúa una condición necesaria y detiene o desvía el flujo cuando no se cumple.
Línea 20	<code>return;</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 21	<code>if (eventData.Payload[0] is not IDictionary<string, object> payload) → Evalúa una condición necesaria y detiene o desvía el flujo cuando no se cumple.</code>
Línea 22	<code>return; → Ejecuta esta instrucción como parte del flujo del ejemplo.</code>
Línea 23	<code>if (!payload.TryGetValue("Name", out var nameValue) nameValue is not string name) → Evalúa una condición necesaria y detiene o desvía el flujo cuando no se cumple.</code>
Línea 24	<code>return; → Ejecuta esta instrucción como parte del flujo del ejemplo.</code>
Línea 26	<code>double? value = null; → Asigna el valor necesario para preparar o registrar el estado de esta operación.</code>
Línea 27	<code>if (payload.TryGetValue("Mean", out var mean) && mean is double meanValue) → Evalúa una condición necesaria y detiene o desvía el flujo cuando no se cumple.</code>
Línea 28	<code>value = meanValue; → Asigna el valor necesario para preparar o registrar el estado de esta operación.</code>
Línea 29	<code>else if (payload.TryGetValue("Increment", out var increment) && increment is double incrementValue) → Forma parte de la implementación concreta del comportamiento explicado en este paso.</code>
Línea 30	<code>value = incrementValue; → Asigna el valor necesario para preparar o registrar el estado de esta operación.</code>
Línea 32	<code>if (value.HasValue) → Evalúa una condición necesaria y detiene o desvía el flujo cuando no se cumple.</code>
Línea 33	<code>_values[name] = value.Value; → Asigna el valor necesario para preparar o registrar el estado de esta operación.</code>
Línea 34	<code>} → Delimita el bloque sintáctico correspondiente.</code>
Línea 35	<code>} → Delimita el bloque sintáctico correspondiente.</code>

Paso 5: Implementar y estudiar `Program.cs`

```
var serilog = new LoggerConfiguration()
    .MinimumLevel.Information()
    .Enrich.FromLogContext()
    .WriteTo.Console()
    .WriteTo.File(
        Path.Combine(logDirectory, "aceria-.log"),
        rollingInterval: RollingInterval.Day,
        fileSizeLimitBytes: 4096,
        rollOnFileSizeLimit: true,
        retainedFileCountLimit: 3,
        shared: false)
    .CreateLogger();

services.AddApplicationInsightsTelemetryWorkerService(options =>
{
    options.ConnectionString = "InstrumentationKey=00000000-0000-0000-0000-000000000000";
});
```

CSHARP

Explicación línea a línea — `Program.cs`

Línea 1	<code>var serilog = new LoggerConfiguration() → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.</code>
Línea 2	<code>.MinimumLevel.Information() → Forma parte de la implementación concreta del comportamiento explicado en este paso.</code>

Línea 3	<code>.Enrich.FromLogContext()</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 4	<code>.WriteTo.Console()</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 5	<code>.WriteTo.File()</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 6	<code>Path.Combine(logDirectory, "aceria-.log")</code> , → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 7	<code>rollingInterval: RollingInterval.Day</code> , → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 8	<code>fileSizeLimitBytes: 4096</code> , → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 9	<code>rollOnFileSizeLimit: true</code> , → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 10	<code>retainedFileCountLimit: 3</code> , → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 11	<code>shared: false)</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 12	<code>.CreateLogger();</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 14	<code>services.AddApplicationInsightsTelemetryWorkerService(options =></code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 15	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 16	<code>options.ConnectionString = "InstrumentationKey=00000000-0000-0000-0000-000000000000";</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 17	<code>});</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.

Paso 6: Compilar y ejecutar

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet ef migrations has-pending-model-changes --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console --configuration Release
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release --no-build
```

POWERSHELL

La ejecución debe finalizar con **5.10 OK**. Si una comprobación interna falla, la aplicación lanza una excepción y el punto no se considera correctamente reproducido.

Paso 7: Observar EventCounters desde otra terminal

Para mantener el proceso vivo unos segundos:

```
$env:ACERIA_COUNTER_WAIT_MS=15000
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release --no-build
```

POWERSHELL

La aplicación imprime el PID. Con ese valor, desde otra terminal:

```
dotnet-counters monitor --process-id <PID> Microsoft.EntityFrameworkCore
```

POWERSHELL

Esta observación usa los EventCounters de EF Core 8. No se atribuyen al curso métricas introducidas en versiones posteriores.

Diagnóstico técnico

La práctica se considera correcta cuando se observan estas condiciones:

- ILogger/Serilog conservan propiedades estructuradas.
- La rotación usa límite de tamaño y retención.
- EfDiagnosticObserver se suscribe solo al listener EF y libera subscriptions.
- EfEventCounterListener lee EventCounters de EF Core 8.
- Application Insights usa integración WorkerService y un canal local en el laboratorio.

Reto resuelto y ampliación

Reto: Ejecuta la aplicación con ACERIA_COUNTER_WAIT_MS y conecta dotnet-counters al PID impreso por la consola.

Para resolverlo, conserva la misma regla del módulo: primero define qué resultado funcional debe mantenerse; después captura SQL, número de comandos, tracking, historial de migraciones o estado transaccional según corresponda. No uses una medición temporal aislada como sustituto de la evidencia técnica.

Errores comunes

- Habilitar EnableSensitiveDataLogging por defecto.
- Suscribirse a DiagnosticListener y no liberar subscriptions.
- Atribuir a EF Core 8 métricas introducidas en versiones posteriores.
- Usar AddApplicationInsightsTelemetry de ASP.NET como si fuera la integración de una consola.

Cierre acumulativo

1. Ejecuta `dotnet build` en la solución del punto.
2. Ejecuta la aplicación y comprueba el marcador `5.10 OK`.
3. Conserva las migraciones existentes; solo 5.2 añade el cambio de modelo oficial de M5.
4. Comprueba que Application continúa sin depender de Entity Framework Core ni de Infrastructure.
5. Antes de avanzar, relaciona el resultado con el SQL, la transacción, el historial o la prueba que lo demuestra.

Resultado esperado

Al finalizar el punto 5.10, AceriaData debe compilar, ejecutarse sobre SQL Server LocalDB y demostrar de forma observable el comportamiento descrito. El marcador final es:

5.10 OK

TEXT

Conexión con el siguiente punto

El siguiente estado es 5.11 y parte directamente de este proyecto completo.

Punto 5.11 — Testing con EF Core

Contexto del proyecto

Este punto continúa directamente `M05/PROYECTO/5.10`. La carpeta `M05/PROYECTO/5.11` contiene la solución completa `AceriaData.sln` y conserva las capas y migraciones existentes. El modelo oficial no cambia en este punto; se conserva la última migración real ya existente.

Objetivo práctico

Separar unit, provider behavior, integración SQL Server y HTTP, usando migraciones reales, Respawn y WebApplicationFactory.

Paso 1: Abrir el estado completo del punto

```
cd M05/PROYECTO/5.11
dotnet restore AceriaData.sln
```

POWERSHELL

La restauración se ejecuta sobre la solución local del punto. No es necesario copiar archivos desde otros puntos: la carpeta ya contiene el estado acumulativo completo.

Paso 2: Identificar los archivos principales

En este punto se trabajan principalmente estos archivos:

- tests/AceriaData.Tests/ProviderBehavior/ProviderBehaviorTests.cs
- tests/AceriaData.Tests/Integration/SqlServerDatabaseFixture.cs
- tests/AceriaData.Tests/Integration/SqlServerIntegrationTests.cs
- tests/AceriaData.Tests/Integration/ApiIntegrationTests.cs

Los archivos se estudian dentro de su capa real. Application conserva sus abstracciones y casos de uso; Infrastructure contiene EF Core y acceso a proveedor; Console compone y ejecuta; los tests se ubican fuera de las capas de producción cuando aparecen en 5.9 y 5.11.

Paso 3: Implementar y estudiar `ProviderBehaviorTests.cs`

```
[Fact]
public async Task InMemory_NoImponeClaveForaneaRelacional()
{
    var options = new DbContextOptionsBuilder<ProbeContext>()
        .UseInMemoryDatabase(Guid.NewGuid().ToString("N"))
        .Options;

    await using var context = new ProbeContext(options);
    context.Children.Add(new ProbeChild { ParentId = 999, Name = "huerfano" });

    var exception = await Record.ExceptionAsync(() => context.SaveChangesAsync());
    Assert.Null(exception);
}

[Fact]
public void Sqlite_NoDebeUsarseParaValidarRowVersionDeSqlServer()
{
    var options = new DbContextOptionsBuilder<ProbeContext>()
        .UseSqlite("Data Source=:memory:")
        .Options;

    using var context = new ProbeContext(options);
    var property = context.Model.FindEntityType(typeof(ProbeParent))!
        .FindProperty(nameof(ProbeParent.Version))!;

    Assert.True(property.IsConcurrencyToken);
    Assert.Equal(ValueGenerated.OnAddOrUpdate, property.ValueGenerated);
    Assert.Equal("BLOB", property.GetColumnType());
}
```

CSHARP

Explicación línea a línea — `ProviderBehaviorTests.cs`

Línea 1	[Fact] → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 2	public async Task InMemory_NoImponeClaveForaneaRelacional() → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 3	{ → Delimita el bloque sintáctico correspondiente.
Línea 4	var options = new DbContextOptionsBuilder<ProbeContext>() → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 5	.UseInMemoryDatabase(Guid.NewGuid().ToString("N")) → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 6	<code>.Options;</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 8	<code>await using var context = new ProbeContext(options);</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 9	<code>context.Children.Add(new ProbeChild { ParentId = 999, Name = "huerfano" });</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 11	<code>var exception = await Record.ExceptionAsync(() => context.SaveChangesAsync());</code> → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.
Línea 12	<code>Assert.Null(exception);</code> → Expresa una condición observable que la prueba exige para considerar correcto el comportamiento.
Línea 13	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 15	<code>[Fact]</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 16	<code>public void Sqlite_NoDebeUsarseParaValidarRowVersionDeSqlServer()</code> → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 17	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 18	<code>var options = new DbContextOptionsBuilder<ProbeContext>()</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 19	<code>.UseSqlite("Data Source=:memory:");</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 20	<code>.Options;</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 22	<code>using var context = new ProbeContext(options);</code> → Importa el espacio de nombres necesario para los tipos y extensiones usados por el archivo, o declara un recurso con liberación automática cuando se usa <code>using var</code> .
Línea 23	<code>var property = context.Model.FindEntityType(typeof(ProbeParent))!</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 24	<code>.FindProperty(nameof(ProbeParent.Version))!</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 26	<code>Assert.True(property.IsConcurrencyToken);</code> → Expresa una condición observable que la prueba exige para considerar correcto el comportamiento.
Línea 27	<code>Assert.Equal(ValueGenerated.OnAddOrUpdate, property.ValueGenerated);</code> → Expresa una condición observable que la prueba exige para considerar correcto el comportamiento.
Línea 28	<code>Assert.Equal("BLOB", property.GetColumnType());</code> → Expresa una condición observable que la prueba exige para considerar correcto el comportamiento.
Línea 29	<code>}</code> → Delimita el bloque sintáctico correspondiente.

Paso 4: Implementar y estudiar `SqlServerDatabaseFixture.cs`

CSHARP

```
public sealed class SqlServerDatabaseFixture : IAsyncLifetime
{
    private readonly string _databaseName =
        "AceriaDB_M5_11_Tests_" + Guid.NewGuid().ToString("N")[..8];
    private Resawner? _resawner;

    public string ConnectionString =>
        $"Server=(localdb)\
\MSSQLLocalDB;Database={_databaseName};Trusted_Connection=True;TrustServerCertificate=True;MultipleActiveResultSets=f
alse;";
}
```

```

public DbContextOptions<AceriaDbContext> CreateOptions() =>
{
    new DbContextOptionsBuilder<AceriaDbContext>()
        .UseSqlServer(ConnectionString,
            sql => sql.MigrationsAssembly(typeof(AceriaDbContext).Assembly.GetName().Name))
        .Options;

    public async Task InitializeAsync()
    {
        await using var context = new AceriaDbContext(CreateOptions());
        await context.Database.MigrateAsync();

        await using var connection = new SqlConnection(ConnectionString);
        await connection.OpenAsync();
        _resawner = await Resawner.CreateAsync(connection, new ResawnerOptions
        {
            DbAdapter = DbAdapter.SqlServer,
            TablesToIgnore = [new Respawn.Graph.Table("__EFMigrationsHistory")]
        });
    }

    public async Task ResetAsync()
    {
        if (_resawner is null) throw new InvalidOperationException("Respawn no inicializado.");
        await using var connection = new SqlConnection(ConnectionString);
        await connection.OpenAsync();
        await _resawner.ResetAsync(connection);
    }
}

```

Explicación línea a línea — SqlServerDatabaseFixture.cs

Línea 1	<code>public sealed class SqlServerDatabaseFixture : IAsyncLifetime</code> → Declara la clase que encapsula la responsabilidad mostrada en este ejemplo.
Línea 2	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 3	<code>private readonly string _databaseName =</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 4	<code>"AceriaDB_M5_11_Tests_" + Guid.NewGuid().ToString("N")[..8];</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 5	<code>private Resawner? _resawner;</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 7	<code>public string ConnectionString =></code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 8	<code> \$"Server=(localdb)\MSSQLLocalDB;Database={_databaseName};Trusted_Connection=True;TrustServerCertificate=True;MultipleActiveResultSets=false;"</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 10	<code>public DbContextOptions<AceriaDbContext> CreateOptions() =></code> → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 11	<code>new DbContextOptionsBuilder<AceriaDbContext>()</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 12	<code>.UseSqlServer(ConnectionString,</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 13	<code>sql => sql.MigrationsAssembly(typeof(AceriaDbContext).Assembly.GetName().Name))</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 14	<code>.Options;</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 16	<code>public async Task InitializeAsync()</code> → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 17	{ → Delimita el bloque sintáctico correspondiente.
Línea 18	await using var context = new AceriaDbContext(CreateOptions()); → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 19	await context.Database.MigrateAsync(); → Aplica la cadena real de migraciones al proveedor de pruebas o despliegue.
Línea 21	await using var connection = new SqlConnection(ConnectionString); → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 22	await connection.OpenAsync(); → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 23	_respawner = await Respawner.CreateAsync(connection, new RespawnerOptions → Inicializa Respawn con el esquema SQL Server de la base de pruebas.
Línea 24	{ → Delimita el bloque sintáctico correspondiente.
Línea 25	DbAdapter = DbAdapter.SqlServer, → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 26	TablesToIgnore = [new Respawn.Graph.Table("__EFMigrationsHistory")] → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 27	}); → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 28	} → Delimita el bloque sintáctico correspondiente.
Línea 30	public async Task ResetAsync() → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 31	{ → Delimita el bloque sintáctico correspondiente.
Línea 32	if (_respawner is null) throw new InvalidOperationException("Respawn no inicializado."); → Evalúa una condición necesaria y detiene o desvía el flujo cuando no se cumple.
Línea 33	await using var connection = new SqlConnection(ConnectionString); → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 34	await connection.OpenAsync(); → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 35	await _respawner.ResetAsync(connection); → Limpia los datos entre pruebas según la configuración de Respawn.
Línea 36	} → Delimita el bloque sintáctico correspondiente.
Línea 37	} → Delimita el bloque sintáctico correspondiente.

Paso 5: Implementar y estudiar `SqlServerIntegrationTests.cs`

CSHARP

```
[Fact]
public async Task RowVersionSqlServer_DetectaConflictoRealEntreDosContextos()
{
    await _fixture.ResetAsync();
    int id;
    await using (var seed = new AceriaDbContext(_fixture.CreateOptions()))
    {
        var orden = new OrdenFabricacion
        {
            NumeroOrden = "OF-TEST-ROWVERSION",
            Cliente = "Inicial",
            Estado = "Pendiente",
            FechaCreacion = DateTime.UtcNow
        };
    }
}
```

```

    };
    seed.OrdenesFabricacion.Add(orden);
    await seed.SaveChangesAsync();
    id = orden.Id;
    Assert.NotEmpty(orden.RowVersion);
}

await using var contextA = new AceriaDbContext(_fixture.CreateOptions());
await using var contextB = new AceriaDbContext(_fixture.CreateOptions());
var a = await contextA.OrdenesFabricacion.SingleAsync(x => x.Id == id);
var b = await contextB.OrdenesFabricacion.SingleAsync(x => x.Id == id);

a.Cliente = "A";
await contextA.SaveChangesAsync();
b.Cliente = "B";
await Assert.ThrowsAsync<DbUpdateConcurrencyException>(() => contextB.SaveChangesAsync());
}

```

Explicación línea a línea — `SqlServerIntegrationTests.cs`

Línea 1	[Fact] → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 2	public async Task RowVersionSqlServer_DetectaConflictoRealEntreDosContextos() → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 3	{ → Delimita el bloque sintáctico correspondiente.
Línea 4	await _fixture.ResetAsync(); → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 5	int id; → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 6	await using (var seed = new AceriaDbContext(_fixture.CreateOptions())) → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 7	{ → Delimita el bloque sintáctico correspondiente.
Línea 8	var orden = new OrdenFabricacion → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 9	{ → Delimita el bloque sintáctico correspondiente.
Línea 10	NumeroOrden = "OF-TEST-ROWVERSION", → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 11	Cliente = "Inicial", → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 12	Estado = "Pendiente", → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 13	FechaCreacion = DateTime.UtcNow → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 14	}; → Delimita el bloque sintáctico correspondiente.
Línea 15	seed.OrdenesFabricacion.Add(orden); → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 16	await seed.SaveChangesAsync(); → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.
Línea 17	id = orden.Id; → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 18	Assert.NotEmpty(orden.RowVersion); → Expresa una condición observable que la prueba exige para considerar correcto el comportamiento.
Línea 19	} → Delimita el bloque sintáctico correspondiente.

Línea 21	<code>await using var contextA = new AceriaDbContext(_fixture.CreateOptions());</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 22	<code>await using var contextB = new AceriaDbContext(_fixture.CreateOptions());</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 23	<code>var a = await contextA.OrdenesFabricacion.SingleAsync(x => x.Id == id);</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 24	<code>var b = await contextB.OrdenesFabricacion.SingleAsync(x => x.Id == id);</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 26	<code>a.Cliente = "A";</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 27	<code>await contextA.SaveChangesAsync();</code> → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.
Línea 28	<code>b.Cliente = "B";</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 29	<code>await Assert.ThrowsAsync<DbUpdateConcurrencyException>(() => contextB.SaveChangesAsync());</code> → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.
Línea 30	<code>}</code> → Delimita el bloque sintáctico correspondiente.

Paso 6: Implementar y estudiar `ApiIntegrationTests.cs`

CSHARP

```
[Fact]
public async Task WebApplicationFactory_UsaApiRealYSqlServerDePruebas()
{
    await _fixture.ResetAsync();
    await using var factory = new AceriaApiFactory(_fixture.ConnectionString);
    using var client = factory.CreateClient();

    var create = await client.PostAsJsonAsync("/api/ordenes", new
    {
        NumeroOrden = "OF-HTTP-511",
        Cliente = "Cliente HTTP"
    });
    Assert.Equal(HttpStatusCode.Created, create.StatusCode);

    var response = await client.GetAsync("/api/ordenes/count");
    response.EnsureSuccessStatusCode();
    var payload = await response.Content.ReadFromJsonAsync<CountResponse>();
    Assert.NotNull(payload);
    Assert.Equal(1, payload.Total);
}
```

Explicación línea a línea — `ApiIntegrationTests.cs`

Línea 1	<code>[Fact]</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 2	<code>public async Task WebApplicationFactory_UsaApiRealYSqlServerDePruebas()</code> → Declara la operación pública que ejecutará o verificará este comportamiento.
Línea 3	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 4	<code>await _fixture.ResetAsync();</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 5	<code>await using var factory = new AceriaApiFactory(_fixture.ConnectionString);</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 6	<code>using var client = factory.CreateClient();</code> → Importa el espacio de nombres necesario para los tipos y extensiones usados por el archivo, o declara un recurso con liberación automática cuando se usa <code>using var</code> .
Línea 8	<code>var create = await client.PostAsJsonAsync("/api/ordenes", new</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 9	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 10	<code>NumeroOrden = "OF-HTTP-511",</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 11	<code>Cliente = "Cliente HTTP"</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 12	<code>});</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 13	<code>Assert.Equal(HttpStatusCode.Created, create.StatusCode);</code> → Expresa una condición observable que la prueba exige para considerar correcto el comportamiento.
Línea 15	<code>var response = await client.GetAsync("/api/ordenes/count");</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 16	<code>response.EnsureSuccessStatusCode();</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 17	<code>var payload = await response.Content.ReadFromJsonAsync<CountResponse>();</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 18	<code>Assert.NotNull(payload);</code> → Expresa una condición observable que la prueba exige para considerar correcto el comportamiento.
Línea 19	<code>Assert.Equal(1, payload.Total);</code> → Expresa una condición observable que la prueba exige para considerar correcto el comportamiento.
Línea 20	<code>}</code> → Delimita el bloque sintáctico correspondiente.

Paso 7: Compilar y ejecutar

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet ef migrations has-pending-model-changes --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console --configuration Release
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release --no-build
dotnet test tests/AceriaData.Tests/AceriaData.Tests.csproj --configuration Release --no-build
```

POWERSHELL

La ejecución debe finalizar con `5.11 OK`. Si una comprobación interna falla, la aplicación lanza una excepción y el punto no se considera correctamente reproducido.

Diagnóstico técnico

La práctica se considera correcta cuando se observan estas condiciones:

- Moq sustituye interfaces de Application sin mockear DbSet.
- InMemory muestra explícitamente que no impone semántica relacional.
- SQLite se usa para diferencias de proveedor, no para validar rowversion SQL Server.
- LocalDB ejecuta la cadena real de migraciones y concurrencia real.
- Respawn limpia datos y conserva `__EFMigrationsHistory`.
- WebApplicationFactory ejecuta endpoints reales contra la base de pruebas.

Reto resuelto y ampliación

Reto: Añade un test HTTP que cree dos órdenes y verifique el count tras un Reset de Respawn.

Para resolverlo, conserva la misma regla del módulo: primero define qué resultado funcional debe mantenerse; después captura SQL, número de comandos, tracking, historial de migraciones o estado transaccional según corresponda. No uses una medición temporal aislada como sustituto de la evidencia técnica.

Errores comunes

- Usar InMemory como sustituto general de SQL Server.
- Afirmar que SQLite reproduce tipos, collation, funciones y rowversion de SQL Server.
- Usar EnsureCreated en lugar de migraciones para la suite de integración.
- Compartir datos entre tests sin aislamiento.

Cierre acumulativo

1. Ejecuta `dotnet build` en la solución del punto.
2. Ejecuta la aplicación y comprueba el marcador `5.11 OK`.
3. Conserva las migraciones existentes; solo 5.2 añade el cambio de modelo oficial de M5.
4. Comprueba que Application continúa sin depender de Entity Framework Core ni de Infrastructure.
5. Antes de avanzar, relaciona el resultado con el SQL, la transacción, el historial o la prueba que lo demuestra.

Resultado esperado

Al finalizar el punto 5.11, AceriaData debe compilar, ejecutarse sobre SQL Server LocalDB y demostrar de forma observable el comportamiento descrito. El marcador final es:

5.11 OK

TEXT

Conexión con el siguiente punto

El siguiente estado es 5.12 y parte directamente de este proyecto completo.

Punto 5.12 — Buenas prácticas y anti-patrones en persistencia empresarial

Contexto del proyecto

Este punto continúa directamente M05/PROYECTO/5.11. La carpeta M05/PROYECTO/5.12 contiene la solución completa AceriaData.sln y conserva las capas y migraciones existentes. El modelo oficial no cambia en este punto; se conserva la última migración real ya existente.

Objetivo práctico

Cerrar el módulo con refactorizaciones before/after medibles: N+1, over-fetching, tracking y traducción cliente/servidor.

Paso 1: Abrir el estado completo del punto

```
cd M05/PROYECTO/5.12
dotnet restore AceriaData.sln
```

POWERSHELL

La restauración se ejecuta sobre la solución local del punto. No es necesario copiar archivos desde otros puntos: la carpeta ya contiene el estado acumulativo completo.

Paso 2: Identificar los archivos principales

En este punto se trabajan principalmente estos archivos:

- `src/AceriaData.Infrastructure/BuenasPracticasAntiPatronesM5Diagnostico.cs`
- `tests/AceriaData.Tests/Integration/BuenasPracticasAntiPatronesM5Tests.cs`

Los archivos se estudian dentro de su capa real. Application conserva sus abstracciones y casos de uso; Infrastructure contiene EF Core y acceso a proveedor; Console compone y ejecuta; los tests se ubican fuera de las capas de producción cuando aparecen en 5.9 y 5.11.

Paso 3: Implementar y estudiar BuenasPracticasAntiPatronesM5Diagnostico.cs

```
private NMasUnoRefactorM5Resultado MedirNMasUno()
{
    _context.ChangeTracker.Clear();
    SqlCommandCounterInterceptor.Instance.Reset();

    var cabeceras = _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.Id)
        .Select(o => new { o.Id, o.NumeroOrden })
        .ToList();

    var planchasNMasUno = 0;
    foreach (var orden in cabeceras)
    {
        planchasNMasUno += _context.PlanchasAcero
            .AsNoTracking()
            .Count(p => p.OrdenId == orden.Id);
    }
    var consultasNMasUno = checked((int)SqlCommandCounterInterceptor.Instance.Count);

    _context.ChangeTracker.Clear();
    SqlCommandCounterInterceptor.Instance.Reset();
    var conInclude = _context.OrdenesFabricacion
        .AsNoTracking()
        .Include(o => o.Planchas)
        .OrderBy(o => o.Id)
        .ToList();
    var consultasInclude = checked((int)SqlCommandCounterInterceptor.Instance.Count);

    _context.ChangeTracker.Clear();
    SqlCommandCounterInterceptor.Instance.Reset();
    var proyectadas = _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.Id)
        .Select(o => new { o.Id, o.NumeroOrden, TotalPlanchas = o.Planchas.Count })
        .ToList();
    var consultasProyeccion = checked((int)SqlCommandCounterInterceptor.Instance.Count);

    var equivalentes =
        cabeceras.Count == conInclude.Count &&
        conInclude.Count == proyectadas.Count &&
        planchasNMasUno == conInclude.Sum(o => o.Planchas.Count) &&
        planchasNMasUno == proyectadas.Sum(o => o.TotalPlanchas);

    return new NMasUnoRefactorM5Resultado(
        cabeceras.Count, planchasNMasUno,
        consultasNMasUno, consultasInclude, consultasProyeccion,
        0, 0, 0, equivalentes, string.Empty, string.Empty);
}
```

Explicación línea a línea — BuenasPracticasAntiPatronesM5Diagnostico.cs

Línea 1	private NMasUnoRefactorM5Resultado MedirNMasUno() → Declara una operación auxiliar usada para mantener aislada la lógica del ejemplo.
Línea 2	{ → Delimita el bloque sintáctico correspondiente.
Línea 3	_context.ChangeTracker.Clear(); → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 4	SqlCommandCounterInterceptor.Instance.Reset(); → Reinicia el contador para que solo se midan los comandos de este escenario.
Línea 6	var cabeceras = _context.OrdenesFabricacion → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 7	.AsNoTracking() → Ejecuta una lectura sin dejar entidades en el ChangeTracker.
Línea 8	.OrderBy(o => o.Id) → Añade orden determinista a la consulta antes de materializar.

Línea 9	<code>.Select(o => new { o.Id, o.NumeroOrden })</code> → Proyecta únicamente la forma de datos necesaria para el resultado.
Línea 10	<code>.ToList();</code> → Materializa o agrega la consulta en este punto, provocando la ejecución contra el proveedor cuando procede.
Línea 12	<code>var planchasNMasUno = 0;</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 13	<code>foreach (var orden in cabeceras)</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 14	<code>{</code> → Delimita el bloque sintáctico correspondiente.
Línea 15	<code>planchasNMasUno += _context.PlanchasAcero</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 16	<code>.AsNoTracking();</code> → Ejecuta una lectura sin dejar entidades en el ChangeTracker.
Línea 17	<code>.Count(p => p.OrdenId == orden.Id);</code> → Materializa o agrega la consulta en este punto, provocando la ejecución contra el proveedor cuando procede.
Línea 18	<code>}</code> → Delimita el bloque sintáctico correspondiente.
Línea 19	<code>var consultasNMasUno = checked((int)SqlCommandCounterInterceptor.Instance.Count);</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 21	<code>_context.ChangeTracker.Clear();</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 22	<code>SqlCommandCounterInterceptor.Instance.Reset();</code> → Reinicia el contador para que solo se midan los comandos de este escenario.
Línea 23	<code>var conInclude = _context.OrdenesFabricacion</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 24	<code>.AsNoTracking();</code> → Ejecuta una lectura sin dejar entidades en el ChangeTracker.
Línea 25	<code>.Include(o => o.Planchas)</code> → Carga la navegación indicada en la consulta relacional para evitar accesos posteriores por fila.
Línea 26	<code>.OrderBy(o => o.Id)</code> → Añade orden determinista a la consulta antes de materializar.
Línea 27	<code>.ToList();</code> → Materializa o agrega la consulta en este punto, provocando la ejecución contra el proveedor cuando procede.
Línea 28	<code>var consultasInclude = checked((int)SqlCommandCounterInterceptor.Instance.Count);</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 30	<code>_context.ChangeTracker.Clear();</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 31	<code>SqlCommandCounterInterceptor.Instance.Reset();</code> → Reinicia el contador para que solo se midan los comandos de este escenario.
Línea 32	<code>var proyectadas = _context.OrdenesFabricacion</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 33	<code>.AsNoTracking();</code> → Ejecuta una lectura sin dejar entidades en el ChangeTracker.
Línea 34	<code>.OrderBy(o => o.Id)</code> → Añade orden determinista a la consulta antes de materializar.
Línea 35	<code>.Select(o => new { o.Id, o.NumeroOrden, TotalPlanchas = o.Planchas.Count })</code> → Proyecta únicamente la forma de datos necesaria para el resultado.
Línea 36	<code>.ToList();</code> → Materializa o agrega la consulta en este punto, provocando la ejecución contra el proveedor cuando procede.

Línea 37	<code>var consultasProyeccion = checked((int)SqlCommandCounterInterceptor.Instance.Count);</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 39	<code>var equivalentes =</code> → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.
Línea 40	<code>cabeceras.Count == conInclude.Count &&</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 41	<code>conInclude.Count == proyectadas.Count &&</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 42	<code>planchasNMasUno == conInclude.Sum(o => o.Planchas.Count) &&</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 43	<code>planchasNMasUno == proyectadas.Sum(o => o.TotalPlanchas);</code> → Asigna el valor necesario para preparar o registrar el estado de esta operación.
Línea 45	<code>return new NMasUnoRefactorM5Resultado(</code> → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.
Línea 46	<code>cabeceras.Count, planchasNMasUno,</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 47	<code>consultasNMasUno, consultasInclude, consultasProyeccion,</code> → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 48	<code>0, 0, 0, equivalentes, string.Empty, string.Empty);</code> → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 49	<code>}</code> → Delimita el bloque sintáctico correspondiente.

Paso 4: Implementar y estudiar `BuenasPracticasAntiPatronesM5Diagnostico.cs`

```
private TraduccionWhereM5Resultado MedirTraduccion()
{
    _context.ChangeTracker.Clear();
    SqlCommandCounterInterceptor.Instance.Reset();

    var falloTraduccion = false;
    try
    {
        _ = _context.OrdenesFabricacion
            .AsNoTracking()
            .Where(o => EsPendiente(o.Estado))
            .ToList();
    }
    catch (InvalidOperationException)
    {
        falloTraduccion = true;
    }

    var comandosAntesCliente = SqlCommandCounterInterceptor.Instance.Count;

    SqlCommandCounterInterceptor.Instance.Reset();
    var pendientesCliente = _context.OrdenesFabricacion
        .AsNoTracking()
        .AsEnumerable()
        .Count(o => EsPendiente(o.Estado));
    var comandosCliente = checked((int)SqlCommandCounterInterceptor.Instance.Count);

    return new TraduccionWhereM5Resultado(
        falloTraduccion,
        checked((int)comandosAntesCliente),
        pendientesCliente > 0,
        comandosCliente,
        pendientesCliente);
}
```

CSHARP

Explicación línea a línea — BuenasPracticasAntiPatronesM5Diagnostico.cs

Línea 1	<code>private TraduccionWhereM5Resultado MedirTraduccion() → Declara una operación auxiliar usada para mantener aislada la lógica del ejemplo.</code>
Línea 2	<code>{ → Delimita el bloque sintáctico correspondiente.</code>
Línea 3	<code>_context.ChangeTracker.Clear(); → Ejecuta esta instrucción como parte del flujo del ejemplo.</code>
Línea 4	<code>SqlCommandCounterInterceptor.Instance.Reset(); → Reinicia el contador para que solo se midan los comandos de este escenario.</code>
Línea 6	<code>var falloTraduccion = false; → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.</code>
Línea 7	<code>try → Abre el bloque en el que se espera que pueda producirse la excepción que se quiere estudiar.</code>
Línea 8	<code>{ → Delimita el bloque sintáctico correspondiente.</code>
Línea 9	<code>_ = _context.OrdenesFabricacion → Asigna el valor necesario para preparar o registrar el estado de esta operación.</code>
Línea 10	<code>.AsNoTracking() → Ejecuta una lectura sin dejar entidades en el ChangeTracker.</code>
Línea 11	<code>.Where(o => EsPendiente(o.Estado)) → Añade el predicado a la consulta; EF Core intentará traducirlo al proveedor antes de materializar.</code>
Línea 12	<code>.ToList(); → Materializa o agrega la consulta en este punto, provocando la ejecución contra el proveedor cuando procede.</code>
Línea 13	<code>} → Delimita el bloque sintáctico correspondiente.</code>
Línea 14	<code>catch (InvalidOperationException) → Captura la excepción específica para aplicar la política de resolución prevista.</code>
Línea 15	<code>{ → Delimita el bloque sintáctico correspondiente.</code>
Línea 16	<code>falloTraduccion = true; → Asigna el valor necesario para preparar o registrar el estado de esta operación.</code>
Línea 17	<code>} → Delimita el bloque sintáctico correspondiente.</code>
Línea 19	<code>var comandosAntesCliente = SqlCommandCounterInterceptor.Instance.Count; → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.</code>
Línea 21	<code>SqlCommandCounterInterceptor.Instance.Reset(); → Reinicia el contador para que solo se midan los comandos de este escenario.</code>
Línea 22	<code>var pendientesCliente = _context.OrdenesFabricacion → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.</code>
Línea 23	<code>.AsNoTracking() → Ejecuta una lectura sin dejar entidades en el ChangeTracker.</code>
Línea 24	<code>.AsEnumerable() → Cruza de forma explícita la frontera de consulta remota a evaluación en memoria.</code>
Línea 25	<code>.Count(o => EsPendiente(o.Estado)); → Materializa o agrega la consulta en este punto, provocando la ejecución contra el proveedor cuando procede.</code>
Línea 26	<code>var comandosCliente = checked((int)SqlCommandCounterInterceptor.Instance.Count); → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.</code>
Línea 28	<code>return new TraduccionWhereM5Resultado(→ Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.</code>

Línea 29	falloTraduccion, → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 30	checked((int)comandosAntesCliente), → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 31	pendientesCliente > 0, → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 32	comandosCliente, → Forma parte de la implementación concreta del comportamiento explicado en este paso.
Línea 33	pendientesCliente); → Ejecuta esta instrucción como parte del flujo del ejemplo.
Línea 34	} → Delimita el bloque sintáctico correspondiente.

Paso 5: Implementar y estudiar BuenasPracticasAntiPatronesM5Tests.cs

CSHARP

```
Assert.True(resultado.NMasUno.ResultadosEquivalentes);
Assert.True(resultado.NMasUno.ConsultasNMasUno > resultado.NMasUno.ConsultasInclude);
Assert.True(resultado.NMasUno.ConsultasNMasUno > resultado.NMasUno.ConsultasProyeccion);
Assert.Equal(0, resultado.NMasUno.TrackingInclude);
Assert.Equal(0, resultado.NMasUno.TrackingProyeccion);

Assert.True(resultado.OverFetching.ResultadosEquivalentes);
Assert.True(resultado.OverFetching.ColumnasEntidadCompleta > resultado.OverFetching.ColumnasProyeccion);
Assert.True(resultado.OverFetching.TrackingEntidadCompleta > 0);
Assert.Equal(0, resultado.OverFetching.TrackingProyeccion);
Assert.True(resultado.OverFetching.SqlProyeccionExcluyeRowVersion);

Assert.True(resultado.Traduccion.MetodoNoTraducibleFalla);
Assert.Equal(0, resultado.Traduccion.ComandosEmitidosAntesDelFallo);
Assert.True(resultado.Traduccion.EvaluacionClienteExplicitaFunciona);
Assert.Equal(1, resultado.Traduccion.ComandosEvaluacionCliente);
```

Explicación línea a línea — BuenasPracticasAntiPatronesM5Tests.cs

Línea 1	Assert.True(resultado.NMasUno.ResultadosEquivalentes); → Expresa una condición observable que la prueba exige para considerar correcto el comportamiento.
Línea 2	Assert.True(resultado.NMasUno.ConsultasNMasUno > resultado.NMasUno.ConsultasInclude); → Expresa una condición observable que la prueba exige para considerar correcto el comportamiento.
Línea 3	Assert.True(resultado.NMasUno.ConsultasNMasUno > resultado.NMasUno.ConsultasProyeccion); → Expresa una condición observable que la prueba exige para considerar correcto el comportamiento.
Línea 4	Assert.Equal(0, resultado.NMasUno.TrackingInclude); → Expresa una condición observable que la prueba exige para considerar correcto el comportamiento.
Línea 5	Assert.Equal(0, resultado.NMasUno.TrackingProyeccion); → Expresa una condición observable que la prueba exige para considerar correcto el comportamiento.
Línea 7	Assert.True(resultado.OverFetching.ResultadosEquivalentes); → Expresa una condición observable que la prueba exige para considerar correcto el comportamiento.
Línea 8	Assert.True(resultado.OverFetching.ColumnasEntidadCompleta > resultado.OverFetching.ColumnasProyeccion); → Expresa una condición observable que la prueba exige para considerar correcto el comportamiento.
Línea 9	Assert.True(resultado.OverFetching.TrackingEntidadCompleta > 0); → Expresa una condición observable que la prueba exige para considerar correcto el comportamiento.
Línea 10	Assert.Equal(0, resultado.OverFetching.TrackingProyeccion); → Expresa una condición observable que la prueba exige para considerar correcto el comportamiento.

Línea 11	<code>Assert.True(resultado.OverFetching.SqlProyeccionExcluyeRowVersion);</code> → Expresa una condición observable que la prueba exige para considerar correcto el comportamiento.
Línea 13	<code>Assert.True(resultado.Traduccion.MetodoNoTraducibleFalla);</code> → Expresa una condición observable que la prueba exige para considerar correcto el comportamiento.
Línea 14	<code>Assert.Equal(0, resultado.Traduccion.ComandosEmitidosAntesDelFallo);</code> → Expresa una condición observable que la prueba exige para considerar correcto el comportamiento.
Línea 15	<code>Assert.True(resultado.Traduccion.EvaluacionClienteExplicitaFunciona);</code> → Expresa una condición observable que la prueba exige para considerar correcto el comportamiento.
Línea 16	<code>Assert.Equal(1, resultado.Traduccion.ComandosEvaluacionCliente);</code> → Expresa una condición observable que la prueba exige para considerar correcto el comportamiento.

Paso 6: Compilar y ejecutar

POWERSHELL

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet ef migrations has-pending-model-changes --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console --configuration Release
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release --no-build
dotnet test tests/AceriaData.Tests/AceriaData.Tests.csproj --configuration Release --no-build
```

La ejecución debe finalizar con `5.12 OK`. Si una comprobación interna falla, la aplicación lanza una excepción y el punto no se considera correctamente reproducido.

Diagnóstico técnico

La práctica se considera correcta cuando se observan estas condiciones:

- N+1, Include y proyección producen resultados funcionalmente equivalentes.
- Se cuentan roundtrips reales.
- Include/proyección de lectura no dejan tracking.
- La proyección materializa menos columnas y excluye RowVersion.
- Un método no traducible en Where falla antes de emitir SQL.
- AsEnumerable hace explícita la frontera de evaluación cliente.

Reto resuelto y ampliación

Reto: Añade una variante con `AsSplitQuery` para dos colecciones, mide sus roundtrips y explica el trade-off frente a una única consulta.

Para resolverlo, conserva la misma regla del módulo: primero define qué resultado funcional debe mantenerse; después captura SQL, número de comandos, tracking, historial de migraciones o estado transaccional según corresponda. No uses una medición temporal aislada como sustituto de la evidencia técnica.

Errores comunes

- Aplicar Include siempre para resolver N+1.
- Aplicar `AsSplitQuery` automáticamente con varias colecciones.
- Tratar Data Annotations como anti-patrón por definición.
- Afirmar que un método personalizado en Where se filtra silenciosamente en memoria.
- Tratar Repository/UoW como requisito universal.

Cierre acumulativo

1. Ejecuta `dotnet build` en la solución del punto.
2. Ejecuta la aplicación y comprueba el marcador `5.12 OK`.
3. Conserva las migraciones existentes; solo 5.2 añade el cambio de modelo oficial de M5.
4. Comprueba que Application continúa sin depender de Entity Framework Core ni de Infrastructure.
5. Antes de avanzar, relaciona el resultado con el SQL, la transacción, el historial o la prueba que lo demuestra.

Resultado esperado

Al finalizar el punto 5.12, AceriaData debe compilar, ejecutarse sobre SQL Server LocalDB y demostrar de forma observable el comportamiento descrito. El marcador final es:

5.12 OK

TEXT

Cierre del módulo

Con 5.12 se completa la secuencia práctica del Módulo 5. El proyecto final contiene concurrencia optimista, resolución de conflictos, transacciones, estrategias de despliegue, scripts idempotentes, trabajo en equipo con migraciones, Repository/UoW como decisión arquitectónica, observabilidad, una estrategia de testing multicapa y refactorizaciones before/after de anti-patrones de persistencia.